claude-windows / f81fb2b5-e34b-427e-a8a1-a37eb4539a49

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f81fb2b5-e34b-427e-a8a1-a37eb4539a49\subagents\workflows\wf_085a5118-2cb\agent-a56d12654a02245c1.jsonl`
- SHA-256: `462034f6161818ef23fe91508017d7019504071a1e80de551ab5ad8bc676dc73`
- Source modified: `2026-06-20T16:58:14+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_085a5118-2cb`
- session_id: `f81fb2b5-e34b-427e-a8a1-a37eb4539a49`

Transcript

1. user (2026-06-20T16:56:24.756Z)

Map per-video artifact write sites for EVAL, REELS/HIGHLIGHTS, RUN LOGS, and DB metadata. Search backend/eval/*, scripts/*, backend/pipeline for: rally_seg_eval / run_regression output dirs (e.g. output/wasb/<clip>_*_wasb.csv), highlight/reel stitching output (extract_video_chunk, highlight reel mp4), where run logs are written, and the DB schema in backend/infrastructure/database.py ? what per-video METADATA and any file PATHS are stored (videos table, candidate_segments, etc.). Record file:line, path patterns, hardcoded?, keyed_by, and note which fields are pure metadata vs file-path references.

2. assistant (2026-06-20T16:56:27.454Z)

I'll help you map the per-video artifact write sites and database schema for this project. Let me start by exploring the codebase structure.

3. user (2026-06-20T16:56:28.108Z)

backend\eval__init__.py
backend\eval\classifier.py
backend\eval\training.py
backend\providers__init__.py
backend\sports__init__.py
backend\sports\badminton.py
backend\sports\base.py
backend\utils__init__.py
backend\utils\geometry.py
scratch\analyze_motion.py
scratch\clean_state.py
scratch\diagnose_rally.py
scratch\gemini_video_debugger.py
scratch\list_models.py
scratch\test_thresholds.py
tests__init__.py
tests\test_eval_batch.py
tests\test_eval_training.py
backend\annotations\base.py
backend\annotations\file_provider.py
tests\test_annotation_providers.py
backend\annotations\automated_provider.py
backend\annotations__init__.py
backend\tools\rally_slicer.py
backend\tools__init__.py
tests\test_calibration.py
tests\test_calibrate_wasb.py
backend\eval\gemini_refine.py
tests\test_export_wasb_segments.py

tests\test_gemini_refine.py
tests\test_calibrate_local.py
tests\test_distill_local.py
backend\eval\audio__init__.py
backend\eval\audio\e1_probe.py
backend\eval\export_wasb_segments.py
backend\pipeline\audio__init__.py
backend\pipeline\validators__init__.py
scratch\test_ai_segmenter.py
tests\test_automated_provider.py
tests\test_eval_harness.py
tests\test_hit_detector.py
run_all_tests.py
run_eval.py
tests\test_hashing.py
tests\test_rally_slicer.py
tests\test_reporting_spec_faq.py
tests\test_eval_partial_labels.py
tests\test_detector_keying.py
backend\eval\metrics.py
backend\eval\regression.py
run_regression.py
tests\test_eval_metrics.py
backend\eval\annotations.py
tests\test_eval_annotations.py
tests\test_gt_loader.py
backend\features__init__.py
tests\test_import_smoke.py
training\gen0__init__.py
training__init__.py
backend\reporting\harvest.py
backend\reporting\run_signals.py
tests\test_reporting_harvest.py
tests\test_run_signals.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\eval_partial_labels.py
backend\eval\rally_seq_proto.py
backend\features\rally_seq.py
backend\pipeline\detectors__init__.py
backend\pipeline\segmenters\base.py
backend\pipeline\validators\blur.py
backend\pipeline\validators\format.py
backend\pipeline\validators\pts_continuity.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\scene_cut.py
backend\providers\base.py
backend\reporting__init__.py
tests\test_eval_classifier.py
tests\test_eval_regression.py
tests\test_geometry.py
tests\test_path_safety.py
tests\test_providers.py
tests\test_rally_seq_proto.py
tests\test_reliability_hardening.py
tests\test_reporting.py
tests\test_sport_profiles.py
tests\test_video.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\pipeline\audio\hit_detector.py
backend\pipeline\validators\base.py
backend\utils\paths.py

```
backend\utils\video.py
backend\pipeline\segmenters\tracknet_hybrid.py
scratch\run_gx010125.py
scratch\probe_cv2_count.py
scratch\probe_gemini.py
scratch\repro_drawtext_font.py
(Results are truncated. Consider using a more specific path or pattern.)
```

4. user (2026-06-20T16:56:28.479Z)

```
total 375
drwxr-xr-x 1 avidu 197609     0 Jun 20 20:48 .
drwxr-xr-x 1 avidu 197609     0 Jun 18 11:14 ..
drwxr-xr-x 1 avidu 197609     0 Jun 20 21:27 .claude
-rw-r--r-- 1 avidu 197609 77824 Jun 20 15:49 .coverage
-rw-r--r-- 1 avidu 197609   611 Jun 11 10:30 .coveragerc
-rw-r--r-- 1 avidu 197609    69 Jun 10 23:25 .env
drwxr-xr-x 1 avidu 197609     0 Jun 20 21:45 .git
-rw-r--r-- 1 avidu 197609   629 Jun 20 13:48 .gitattributes
drwxr-xr-x 1 avidu 197609     0 Jun 10 11:31 .github
-rw-r--r-- 1 avidu 197609   924 Jun 15 14:11 .gitignore
drwxr-xr-x 1 avidu 197609     0 Jun 11 10:42 .mypy_cache
drwxr-xr-x 1 avidu 197609     0 May 19 15:58 .pytest_cache
drwxr-xr-x 1 avidu 197609     0 Jun 11 10:32 .ruff_cache
drwxr-xr-x 1 avidu 197609     0 Jun  8 07:11 .sixth
-rw-r--r-- 1 avidu 197609  4743 Jun 19 21:46 CLAUDE.md
-rw-r--r-- 1 avidu 197609 33528 Jun 15 14:11 README.md
drwxr-xr-x 1 avidu 197609     0 Jun 14 01:27 __pycache__
drwxr-xr-x 1 avidu 197609     0 Jun 20 14:42 artifacts
drwxr-xr-x 1 avidu 197609     0 Jun 20 13:48 backend
-rw-r--r-- 1 avidu 197609  2735 Jun 16 01:52 config.json
drwxr-xr-x 1 avidu 197609     0 Jun 20 14:51 deploy
drwxr-xr-x 1 avidu 197609     0 Jun 20 19:42 docs
drwxr-xr-x 1 avidu 197609     0 Jun 19 21:21 eval_baselines
-rw-r--r-- 1 avidu 197609  2344 Jun 20 13:48 mypy.ini
drwxr-xr-x 1 avidu 197609     0 Jun 20 15:49 output
-rw-r--r-- 1 avidu 197609  3897 Jun 20 15:33 pyproject.toml
-rw-r--r-- 1 avidu 197609   240 May 30 14:08 pytest.ini
-rw-r--r-- 1 avidu 197609   629 Jun  7 14:14 requirements.txt
-rw-r--r-- 1 avidu 197609   884 Jun 14 12:10 ruff.toml
-rwxr-xr-x 1 avidu 197609  2055 Jun  7 14:52 run_all_tests.py
-rw-r--r-- 1 avidu 197609  5620 Jun  8 07:11 run_eval.py
-rw-r--r-- 1 avidu 197609  4356 Jun 10 13:45 run_regression.py
drwxr-xr-x 1 avidu 197609     0 Jun 20 21:56 scratch
drwxr-xr-x 1 avidu 197609     0 Jun 20 13:48 scripts
drwxr-xr-x 1 avidu 197609     0 Jun 20 19:38 tests
drwxr-xr-x 1 avidu 197609     0 Jun 19 14:26 tools
drwxr-xr-x 1 avidu 197609     0 Jun 10 15:57 training
```

5. user (2026-06-20T16:56:28.726Z)

```
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/scripts
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/backend
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-
execpolicy/backend/eval
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-execpolicy/scripts
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/boxhill17/backend
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/boxhill17/backend/eval
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/boxhill17/scripts
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/gap-doc/backend
```

```
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/gap-doc/backend/eval
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/gap-doc/scripts
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/hardchop/backend
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/hardchop/backend/eval
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/hardchop/scripts
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/pr-stitch/backend
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/pr-stitch/backend/eval
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/pr-stitch/scripts
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/recdoc/backend
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/recdoc/backend/eval
```

6. assistant (2026-06-20T16:56:30.702Z)

Now let me search for the database schema and key evaluation/pipeline files:

7. user (2026-06-20T16:56:31.885Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend/infrastructure
```

8. user (2026-06-20T16:56:31.957Z)

```
backend\infrastructure\execution_policy.py
backend\infrastructure\database.py
```

9. user (2026-06-20T16:56:32.019Z)

```
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/agent-a771ff1d85e18f2ed/backend/infrastructure/database.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/bhi-
execpolicy/backend/infrastructure/database.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/boxhill17/backend/infrastructure/database.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/gap-
doc/backend/infrastructure/database.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/hardchop/backend/infrastructure/database.py
C:\Users\avidu\Projects\badminton-highlight-indexer/.claude/worktrees/pr-
stitch/backend/infrastructure/database.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/recdoc/backend/infrastructure/database.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/resweep/backend/infrastructure/database.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/tier1-windowing/backend/infrastructure/database.py
C:\Users\avidu\Projects\badminton-highlight-
indexer/.claude/worktrees/w2-safeguard/backend/infrastructure/database.py
C:\Users\avidu\Projects\badminton-highlight-indexer/backend/infrastructure/database.py
```

10. assistant (2026-06-20T16:56:33.832Z)

Good. Now let me read the key files and search for artifact write patterns:

11. user (2026-06-20T16:56:34.280Z)

```
1       """CLI to evaluate detected rallies against hand-annotated ground truth.
2
3       Scores predictions already stored in the SQLite DB (run the pipeline first) ? no
4       API calls, fully deterministic. See docs/EVALUATION.md for the workflow.
5
6       Examples
7       --------
8       Create an empty annotation file to fill in:
9           python run_eval.py --scaffold --annotations rallies.csv
10
```

```
    Score a video's detected rallies (identify it by file or by stored video_id):
        python run_eval.py --video "C:/path/match.mp4" --annotations rallies.csv
        python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5

    Show which rallies were missed / spurious, and dump a JSON report:
        python run_eval.py --video-id <md5> --annotations rallies.csv --show-failures --json
report.json
    """

    import os
    import sys
    import json
    import argparse
    import logging

    from backend.infrastructure.database import Database
    from backend.config import load_config
    from backend.utils.hashing import compute_video_id as get_file_md5  # canonical video_id
    from backend.eval.annotations import load_annotations, write_scaffold
    from backend.eval.harness import evaluate, format_report_text, report_to_dict, STAGES


    def _default_db_path() -> str:
        """Resolve the DB path from the typed config (output.output_dir /
output.db_name)."""
        cfg = load_config()
        return os.path.join(cfg.output.output_dir, cfg.output.db_name)


    _EPILOG = """\
    Prerequisite:
      The harness scores predictions read from the SQLite DB ? it does NOT run the
      pipeline. So you must first PROCESS the video (e.g. via run_process_and_stitch.py)
      so its detections land in output/sports_indexer.db. Then point this tool at the
      same video file (--video) or its stored MD5 (--video-id). If you get
      "database not found" or zero predictions, the video hasn't been processed yet.

    See docs/EVALUATION.md for the annotation format and how to read the output.
    """


    def build_parser() -> argparse.ArgumentParser:
        p = argparse.ArgumentParser(
            description="Evaluate detected rallies vs ground-truth annotations.",
            epilog=_EPILOG,
            formatter_class=argparse.RawDescriptionHelpFormatter,
        )
        p.add_argument("--annotations", help="Path to the ground-truth rally CSV (start,end
per row).")
        p.add_argument("--video", help="Path to the video file (its MD5 becomes the
video_id).")
        p.add_argument("--video-id", help="Stored video_id (MD5) to evaluate, instead of
--video.")
        p.add_argument("--db", default=None, help="SQLite DB path (default: from
config.json).")
        p.add_argument("--stage", choices=("candidates", "final", "both"), default="both",
                       help="Which prediction stage(s) to score (default: both).")
        p.add_argument("--detector", default=None,
                       help="Only score candidates from this detector (e.g. yolo_hybrid,
tracknet_hybrid).")
        p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
        p.add_argument("--pr-curve", action="store_true", help="Include an IoU sweep
(0.1..0.9) in the report.")
        p.add_argument("--show-failures", action="store_true",
```

```
                        help="List missed (FN) and spurious (FP) rallies with timestamps.")
    p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report to
this path.")
    p.add_argument("--scaffold", action="store_true",
                        help="Write an empty annotation CSV to --annotations and exit.")
    return p


def main(argv=None) -> int:
    logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
    args = build_parser().parse_args(argv)

    # Scaffold mode: just write the template and exit.
    if args.scaffold:
        if not args.annotations:
            print("ERROR: --scaffold requires --annotations <path>", file=sys.stderr)
            return 2
        write_scaffold(args.annotations)
        print(f"Wrote empty annotation file: {args.annotations}")
        return 0

    if not args.annotations:
        print("ERROR: --annotations is required (or use --scaffold).", file=sys.stderr)
        return 2

    # Resolve the video_id.
    if args.video_id:
        video_id = args.video_id
    elif args.video:
        if not os.path.exists(args.video):
            print(f"ERROR: video not found: {args.video}", file=sys.stderr)
            return 2
        print("Computing video MD5...")
        video_id = get_file_md5(args.video)
        print(f"video_id = {video_id}")
    else:
        print("ERROR: provide --video <path> or --video-id <md5>.", file=sys.stderr)
        return 2

    db_path = args.db or _default_db_path()
    if not os.path.exists(db_path):
        print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
        return 2

    gt = load_annotations(args.annotations)
    if not gt:
        print(f"WARNING: no ground-truth rallies in {args.annotations}.",
file=sys.stderr)

    stages = list(STAGES) if args.stage == "both" else [args.stage]
    db = Database(db_path)
    report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
                      detector=args.detector, with_pr_curve=args.pr_curve)

    print(format_report_text(report, show_failures=args.show_failures))

    if args.json_out:
        with open(args.json_out, "w") as f:
            json.dump(report_to_dict(report), f, indent=2)
        print(f"\nJSON report written to {args.json_out}")

    return 0


```

```
129     if __name__ == "__main__":
130         raise SystemExit(main())
131
```

12. user (2026-06-20T16:56:34.510Z)

```
1       """CLI: regression-test the production tool against golden-set ground truth.
2
3       Obtains ground truth from an annotation provider (a tier), scores the video's
4       stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5       detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7       Examples
8       --------
9       Snapshot the current numbers as the baseline (first run / after a real improvement):
10          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12      Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13          python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14      """
15      import os
16      import sys
17      import json
18      import argparse
19      import logging
20
21      from backend.infrastructure.database import Database
22      from backend.config import load_config
23      from backend.annotations import annotation_registry
24      from backend.eval import regression
25
26
27      def _default_db_path() -> str:
28          cfg = load_config()
29          return os.path.join(cfg.output.output_dir, cfg.output.db_name)
30
31
32      def build_parser() -> argparse.ArgumentParser:
33          p = argparse.ArgumentParser(description="Regression-test the tool vs golden-set
ground truth.")
34          p.add_argument("--video-id", required=True, help="Stored video_id (MD5) whose
predictions to score.")
35          p.add_argument("--tier", default="file",
36                         help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}")
37          p.add_argument("--annotations", help="video_ref for the provider (e.g. CSV path for
the 'file' tier).")
38          p.add_argument("--stage", choices=("candidates", "final"), default="final")
39          p.add_argument("--detector", default=None, help="Restrict candidate scoring to one
detector.")
40          p.add_argument("--iou", type=float, default=0.5)
41          p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
42          p.add_argument("--db", default=None)
43          p.add_argument("--baseline", default=regression.DEFAULT_BASELINE_PATH,
help="Baseline JSON path.")
44          p.add_argument("--update-baseline", action="store_true",
45                         help="Snapshot the current numbers as the new baseline instead of
checking.")
46          p.add_argument("--allow-missing-baseline", action="store_true",
47                         help="Treat a missing baseline as a pass (exit 0) instead of a gate
error (exit 3).")
48          p.add_argument("--json", dest="json_out", default=None)
49          return p
50
```

```
51
52      def main(argv=None) -> int:
53          logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
54          args = build_parser().parse_args(argv)
55
56          db_path = args.db or _default_db_path()
57          if not os.path.exists(db_path):
58              print(f"ERROR: database not found at {db_path}. Run the pipeline first.",
file=sys.stderr)
59              return 2
60          db = Database(db_path)
61
62          result = regression.regress_with_provider(
63              db, video_id=args.video_id, video_ref=args.annotations, tier=args.tier,
64              stage=args.stage, iou_threshold=args.iou, detector=args.detector,
65              tolerance=args.tolerance, baseline_path=args.baseline,
66          )
67
68          print(regression.format_result(result))
69
70          if args.update_baseline:
71              regression.save_baseline(result.video_id, result.stage, result.precision,
72                                       result.recall, result.f1, path=args.baseline,
73                                       iou_threshold=result.iou_threshold,
detector=args.detector,
74                                       gt_fingerprint=result.gt_hash)
75              print(f"Baseline updated in {args.baseline}.")
76
77          if args.json_out:
78              with open(args.json_out, "w") as f:
79                  json.dump(result.as_dict(), f, indent=2)
80              print(f"JSON written to {args.json_out}")
81
82          # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
83          if result.regressed:
84              return 1
85          if not result.has_baseline and not args.update_baseline and not
args.allow_missing_baseline:
86              print("ERROR: no baseline to compare against ? the gate cannot verify anything.
"
87                    "Snapshot one with --update-baseline (then commit it), or pass "
88                    "--allow-missing-baseline to accept this run.", file=sys.stderr)
89              return 3
90          return 0
91
92
93      if __name__ == "__main__":
94          raise SystemExit(main())
95
```

13. user (2026-06-20T16:56:34.852Z)

```
1       import sqlite3
2       import json
3       import logging
4       from contextlib import contextmanager
5       from typing import List, Dict, Any, Optional, Tuple
6
7       logger = logging.getLogger(__name__)
8
9       class Database:
10          def __init__(self, db_path: str):
11              self.db_path = db_path
12              self._init_db()
13
```

```
        @contextmanager
        def _connect(self):
            """Transaction scope that also CLOSES the connection.

            ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
            never closes, so every call leaked its connection to GC (lingering file
            handles on the WAL sidecars under Windows). Internal methods use this.
            """
            conn = self._get_connection()
            try:
                with conn:
                    yield conn
            finally:
                conn.close()

        def _get_connection(self):
            # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
            # adds concurrent writers (a swallowed 'database is locked' silently drops
segments).
            conn = sqlite3.connect(self.db_path, timeout=30.0)
            conn.row_factory = sqlite3.Row
            # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
            conn.execute("PRAGMA foreign_keys = ON")
            # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
            conn.execute("PRAGMA busy_timeout = 30000")
            try:
                conn.execute("PRAGMA journal_mode = WAL")
            except sqlite3.OperationalError:
                pass  # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
            return conn

        def _execute_write(self, sql: str, params: Tuple[Any, ...], error_msg: str) -> bool:
            """Run ONE bool-returning single-statement write inside a connection scope.

            Captures the connect/commit/except-logger.error/return-False boilerplate shared
by
            the bool-returning single-statement writers. ``error_msg`` is the caller's exact
            per-method message (already formatted with its own ids) ? logged verbatim on a
            swallowed write fault so each writer keeps its specific log text. Returns True
on
            success, False (after logging ``error_msg``) on any exception."""
            try:
                with self._connect() as conn:
                    conn.cursor().execute(sql, params)
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"{error_msg}: {e}")
                return False

        @staticmethod
        def _add_column_idempotent(cursor: sqlite3.Cursor, ddl: str) -> None:
            """Apply a bare ``ALTER TABLE ... ADD COLUMN`` migration step re-runnably.

            A DDL ``ALTER`` auto-commits independently of the surrounding ``with conn:``
            transaction, so an interrupted v3/v4 block can leave the column added while
            ``user_version`` stays un-bumped ? and the next open would re-run the same
            ``ALTER`` and raise ``OperationalError: duplicate column name``, bricking every
            future ``Database(...)`` construction (``_init_db`` runs in ``__init__``).
Swallow
            ONLY that specific error so the ladder stays crash-safe, matching the re-
runnable
            ``CREATE ... IF NOT EXISTS`` v1/v2/v5 blocks.
            """
            try:
```

```
                cursor.execute(ddl)
            except sqlite3.OperationalError as exc:
                if "duplicate column name" not in str(exc).lower():
                    raise

        def _init_db(self):
            """Initializes tables for videos, play_segments, and events."""
            with self._connect() as conn:
                cursor = conn.cursor()

                self._create_base_tables(cursor)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    self._migrate_to_v1(cursor)

                if version < 2:
                    self._migrate_to_v2(cursor)

                if version < 3:
                    self._migrate_to_v3(cursor)

                if version < 4:
                    self._migrate_to_v4(cursor)

                if version < 5:
                    self._migrate_to_v5(cursor)
                # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6

                conn.commit()

        def _create_base_tables(self, cursor):
            """Create the videos / play_segments / events base tables (idempotent)."""
            # Videos Table
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS videos (
                    id TEXT PRIMARY KEY,
                    filepath TEXT NOT NULL,
                    processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    status TEXT NOT NULL,
                    validation_results TEXT
                )
            """)

            # Play Segments Table (Extensible metadata JSON)
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS play_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    server TEXT,
                    receiver TEXT,
                    metadata TEXT,
                    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                )
            """)

```

```
            # Events Table
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS events (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    segment_id INTEGER NOT NULL,
                    timestamp REAL NOT NULL,
                    type TEXT NOT NULL,
                    player TEXT,
                    details TEXT,
                    FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
                )
            """)

        def _migrate_to_v1(self, cursor):
            # v1: raw candidate detections (pre-confirmation), detector-agnostic.
            # Common fields are columns; detector-specific metrics go in `stats` JSON,
            # so a new segmenter reuses this table by setting its own `detector`/`stats`.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS candidate_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    detector TEXT NOT NULL,
                    chunk_index INTEGER,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    confidence REAL,
                    stats TEXT,
                    detection_params TEXT,
                    confirmed_segment_id INTEGER,
                    human_label TEXT,
                    notes TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                    FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
                )
            """)
            cursor.execute("""
                CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                    ON candidate_segments(video_id, detector)
            """)
            cursor.execute("PRAGMA user_version = 1")

        def _migrate_to_v2(self, cursor):
            # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
            # /api/process request. `status` is the EXECUTION state of the job
            # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
            # that failed validation) lives inside `result` JSON as result["status"].
            # `result` is the full sync-era response dict (incl. report paths), so the
            # observability report is the job's artifact, per the plan.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS jobs (
                    id TEXT PRIMARY KEY,
                    video_path TEXT NOT NULL,
                    video_id TEXT,
                    segmenter TEXT,
                    player_pool TEXT,
                    max_frames INTEGER,
                    status TEXT NOT NULL DEFAULT 'queued',
                    progress TEXT,
                    error TEXT,
                    result TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    started_at TIMESTAMP,
```

```
                    finished_at TIMESTAMP
                )
            """)
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
            cursor.execute("PRAGMA user_version = 2")

        def _migrate_to_v3(self, cursor):
            # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
            # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
due
            # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
            # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
            # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
            # ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
            self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
            cursor.execute("PRAGMA user_version = 3")

        def _migrate_to_v4(self, cursor):
            # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
            # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
            # path can scope rows to an owner. **NULL = local install = byte-identical to
            # today** ? every existing row stays NULL and every existing query (which never
            # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
            # the ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
            # Additive ONLY: no served-behavior change rides on this slice.
            self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
            self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
            # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
            # of index churn while making the eventual per-user lookups cheap.
            cursor.execute(
                "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
                "WHERE user_id IS NOT NULL")
            cursor.execute(
                "CREATE INDEX IF NOT EXISTS idx_candidates_user "
                "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
            cursor.execute("PRAGMA user_version = 4")

        def _migrate_to_v5(self, cursor):
            # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
            # version): the resolved per-user `fusion_config` overlay + an INLINE
            # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
            # and provenance that earned it. `is_active` pins the one row the serving bridge
            # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
            # only persists/loads it ? NOTHING reads it on the served path yet.
            #
            #   user_id              ? owner of the model (NULL allowed = the local install)
            #   version              ? monotonic per-user model version
            #   baseline_version     ? the shared baseline this was fit against (upgrade
switch)
            #   feature_schema_hash  ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
            #   fusion_config        ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
            #   classifier_json      ? JSON LogisticRegression.to_dict() (None = config-only
model)
```

```
253             #   promotion_evidence   ? JSON per_user_gate PromotionDecision (why it was
promoted)
254             #   n_videos_at_fit      ? held-out-user corpus size when fit (min_n trust floor
input)
255             #   is_active            ? the one resolved row for this user (partial-unique
below)
256             cursor.execute("""
257                 CREATE TABLE IF NOT EXISTS user_models (
258                     id INTEGER PRIMARY KEY AUTOINCREMENT,
259                     user_id TEXT,
260                     version INTEGER NOT NULL DEFAULT 1,
261                     baseline_version TEXT,
262                     feature_schema_hash TEXT,
263                     fusion_config TEXT,
264                     classifier_json TEXT,
265                     promotion_evidence TEXT,
266                     n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
267                     is_active INTEGER NOT NULL DEFAULT 0,
268                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
269                 )
270             """)
271             # One model version per user (NULLs compare distinct in SQLite, so the local
272             # install can still carry multiple versioned rows; the active-row guard below
273             # is what enforces a single served model).
274             cursor.execute("""
275                 CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
276                     ON user_models(user_id, version)
277             """)
278             # At most ONE active model per user: a partial-unique index over the active
rows.
279             # (SQLite treats NULL user_id rows as distinct, so the local install's single
280             # active row is still guarded ? there is one local user.)
281             cursor.execute("""
282                 CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
283                     ON user_models(user_id) WHERE is_active = 1
284             """)
285             cursor.execute("PRAGMA user_version = 5")
286
287         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
288             """Register or update a video row WITHOUT touching its child segments.
289
290             Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
291             with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
292             That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
293             before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
294             row in place; clearing segments is now an explicit, deliberate operation
295             (clear_video_data / prune_segments).
296             """
297             return self._execute_write(
298                 "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?,
?, ?) "
299                 "ON CONFLICT(id) DO UPDATE SET "
300                 "filepath=excluded.filepath, status=excluded.status, "
301                 "validation_results=excluded.validation_results",
302                 (video_id, filepath, status, json.dumps(validation_results)),
303                 "Failed to add video",
304             )
305
306         def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
307             """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
```

```
308
309             Captured before a (re)segmentation so the run's new rows (id > watermark) can be
310             told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
311             """
312             with self._connect() as conn:
313                 cur = conn.cursor()
314                 p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
315                                 (video_id,)).fetchone()[0]
316                 c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
317                                 (video_id,)).fetchone()[0]
318                 return int(p), int(c)
319
320         def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
321                            play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
322                            cand_after: Optional[int] = None) -> None:
323             """Delete play/candidate segments by id range (events cascade from
play_segments).
324
325             Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
326             successful run (``*_upto`` = the pre-run watermark), or drop the NEW partial
rows
327             and keep the OLD ones when the run failed/produced nothing (``*_after``).
328             """
329             with self._connect() as conn:
330                 cur = conn.cursor()
331                 if play_upto is not None:
332                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
333                 if play_after is not None:
334                     cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
335                 if cand_upto is not None:
336                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
337                 if cand_after is not None:
338                     cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
339                 conn.commit()
340
341         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
342             try:
343                 with self._connect() as conn:
344                     cursor = conn.cursor()
345                     cursor.execute(
346                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
347                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
348                     )
349                     conn.commit()
350                     return cursor.lastrowid
351             except Exception as e:
352                 logger.error(f"Failed to add play segment: {e}")
353                 return None
354
355         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
356             return self._execute_write(
357                 "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
```

```
                (segment_id, timestamp, event_type, player, json.dumps(details or {})),
                "Failed to add event",
            )

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._connect() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            return self._execute_write(
                "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id = ?",
                (segment_id, candidate_id),
                f"Failed to link candidate {candidate_id} to segment {segment_id}",
            )

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._connect() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates
```

```

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._connect() as conn:
                cursor = conn.cursor()
                rows = cursor.execute(query, params).fetchall()

                for row in rows:
                    r_dict = dict(row)
                    r_id = r_dict["id"]
                    r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                    # Fetch associated events
                    event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
```

```
            with self._connect() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()

        # --- Async job store (DEFERRED_HARDENING_PLAN #16) -------------------------------
        # Writers follow the repo convention (swallow + logger.error, return bool); the
        # worker logs loudly on a failed transition so a job can't silently wedge.

        def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
                       player_pool: Optional[List[str]] = None,
                       max_frames: Optional[int] = None,
                       policy: Optional[Dict[str, Any]] = None) -> bool:
            """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
            (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
            return self._execute_write(
                "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,
status, policy) "
                "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
                (job_id, video_path, segmenter,
                 json.dumps(player_pool) if player_pool is not None else None,
                 max_frames, json.dumps(policy) if policy is not None else None),
                f"Failed to create job {job_id}",
            )

        def mark_job_running(self, job_id: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
                "progress='starting' WHERE id = ?", (job_id,),
                f"Failed to mark job {job_id} running",
            )

        def set_job_progress(self, job_id: str, progress: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id),
                f"Failed to set progress for job {job_id}",
            )

        def set_job_video_id(self, job_id: str, video_id: str) -> bool:
            return self._execute_write(
                "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id),
                f"Failed to set video_id for job {job_id}",
            )

        def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
            """Job EXECUTED to completion. The pipeline's own verdict (success/failed
            validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
            return self._execute_write(
                "UPDATE jobs SET status='done', progress='finished', result=?, "
                "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
                (json.dumps(result), job_id),
                f"Failed to mark job {job_id} done",
            )

        def mark_job_failed(self, job_id: str, error: str) -> bool:
            """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
            return self._execute_write(
                "UPDATE jobs SET status='failed', error=?, "
                "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id),
                f"Failed to mark job {job_id} failed",
            )

        def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
            """Fetch one job; `result`/`player_pool` JSON deserialized."""
```

```
            with self._connect() as conn:
                row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
(job_id,)).fetchone()
                if not row:
                    return None
                d = dict(row)
                d["result"] = json.loads(d["result"]) if d["result"] else None
                d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
None
                d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
                return d

        # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
------------------------
        def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
            """Attach/replace a job's JSON ExecutionPolicy."""
            return self._execute_write(
                "UPDATE jobs SET policy=? WHERE id = ?",
                (json.dumps(policy) if policy is not None else None, job_id),
                f"Failed to set policy for job {job_id}",
            )

        def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
None) -> bool:
            """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
            worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
            computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
            return self._execute_write(
                "UPDATE jobs SET status='waiting', "
                "next_poll_at=datetime('now', ?), "
                "progress=COALESCE(?, progress) WHERE id = ?",
                (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
                f"Failed to set job {job_id} waiting",
            )

        def seconds_until_next_due(self) -> Optional[float]:
            """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
            schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
            already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
            try:
                with self._connect() as conn:
                    row = conn.cursor().execute(
                        "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
                        "  (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
                        "FROM jobs WHERE status='waiting'").fetchone()
                if row is None or row["secs"] is None:
                    return None
                return max(0.0, float(row["secs"]))
            except Exception as e:
                logger.error(f"Failed to compute next-due delay: {e}")
                return None

        def claim_due_job(self) -> Optional[Dict[str, Any]]:
            """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
            flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
            double-claim (the DB row-claim is the only coordination needed; no master).
```

```
Earliest-due
592             first. Returns the claimed job dict, or None if nothing is runnable."""
593             try:
594                 with self._connect() as conn:
595                     cur = conn.cursor()
596                     row = cur.execute(
597                         "SELECT id, status FROM jobs WHERE status='queued' "
598                         "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
599                         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
600                     if not row:
601                         return None
602                     cur.execute(
603                         "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
604                         "WHERE id=? AND status=?", (row["id"], row["status"]))
605                     conn.commit()
606                     if cur.rowcount != 1:
607                         return None                     # lost the race to a concurrent
worker
608                 return self.get_job(row["id"])
609             except Exception as e:
610                 logger.error(f"Failed to claim a due job: {e}")
611                 return None
612
613         def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
614             """Crash recovery: any job still queued/running from a previous process is dead
615             (the executor is in-process). Mark failed so clients see a terminal state, not a
616             hang. Called once at startup. Returns the number of jobs swept."""
617             try:
618                 with self._connect() as conn:
619                     cur = conn.cursor()
620                     cur.execute(
621                         "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
622                         "WHERE status IN ('queued', 'running')", (reason,))
623                     conn.commit()
624                     return cur.rowcount
625             except Exception as e:
626                 logger.error(f"Failed to sweep stale jobs: {e}")
627                 return 0
628
629         # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) ------------------------
630         # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
631         # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
632         # serving.py) resolves them, and wiring that bridge into the production decision
seam
633         # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
634         # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
635         # JSON-serialised dicts; the rest are scalar columns.
636
637         def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
638                               baseline_version: Optional[str] = None,
639                               feature_schema_hash: Optional[str] = None,
640                               fusion_config: Optional[Dict[str, Any]] = None,
641                               classifier_json: Optional[Dict[str, Any]] = None,
642                               promotion_evidence: Optional[Dict[str, Any]] = None,
643                               n_videos_at_fit: int = 0,
644                               is_active: bool = False) -> Optional[int]:
645             """Insert or replace ONE per-user model row keyed by ``(user_id, version)``.
646
647             Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
the
648             rowid). When ``is_active`` is True this row becomes the user's single active
model ?
```

```
            any previously-active row for the SAME user is first cleared, so the
            ``idx_user_models_one_active`` partial-unique invariant always holds (one active
model
            per user). Returns the row id, or None on failure.
            """
            try:
                with self._connect() as conn:
                    cur = conn.cursor()
                    if is_active:
                        # Demote the current active row first (NULL user_id = local: IS NULL
match).
                        if user_id is None:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
                                (version,))
                        else:
                            cur.execute(
                                "UPDATE user_models SET is_active = 0 "
                                "WHERE user_id = ? AND is_active = 1 AND version != ?",
                                (user_id, version))
                    cur.execute(
                        """INSERT INTO user_models
                           (user_id, version, baseline_version, feature_schema_hash,
                            fusion_config, classifier_json, promotion_evidence,
                            n_videos_at_fit, is_active)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
                           ON CONFLICT(user_id, version) DO UPDATE SET
                             baseline_version=excluded.baseline_version,
                             feature_schema_hash=excluded.feature_schema_hash,
                             fusion_config=excluded.fusion_config,
                             classifier_json=excluded.classifier_json,
                             promotion_evidence=excluded.promotion_evidence,
                             n_videos_at_fit=excluded.n_videos_at_fit,
                             is_active=excluded.is_active""",
                        (user_id, version, baseline_version, feature_schema_hash,
                         json.dumps(fusion_config) if fusion_config is not None else None,
                         json.dumps(classifier_json) if classifier_json is not None else
None,
                         json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
                         n_videos_at_fit, 1 if is_active else 0))
                    conn.commit()
                    rid = cur.execute(
                        "SELECT id FROM user_models WHERE "
                        + ("user_id IS NULL" if user_id is None else "user_id = ?")
                        + " AND version = ?",
                        ((version,) if user_id is None else (user_id, version))).fetchone()
                    return int(rid[0]) if rid else None
            except Exception as e:
                logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
                return None

        def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
            """Return the single active per-user model row for ``user_id`` (NULL = local
install),
            with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
            or None if the user has no active model. Does not touch the served pipeline."""
            with self._connect() as conn:
                if user_id is None:
                    row = conn.cursor().execute(
                        "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
```

```
706                 else:
707                     row = conn.cursor().execute(
708                         "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
709                         (user_id,)).fetchone()
710                 if not row:
711                     return None
712                 d = dict(row)
713                 for col in ("fusion_config", "classifier_json", "promotion_evidence"):
714                     d[col] = json.loads(d[col]) if d[col] else None
715                 return d
716
```

14. user (2026-06-20T16:56:35.534Z)

```
Found 16 files
backend\eval\rally_seg_eval.py
backend\eval\golden_real_fixtures.py
backend\eval\golden_fixtures.py
backend\eval\serve_contrast.py
backend\eval\distill_local.py
backend\eval\served_gate_a.py
backend\eval\fusion_golden.py
backend\eval\calibrate_local.py
backend\eval\gt_loader.py
backend\eval\rally_seq_proto.py
backend\eval\eval_partial_labels.py
backend\eval\calibrate_wasb.py
backend\eval\annotations.py
backend\eval\export_wasb_segments.py
backend\eval\audio\e1_probe.py
backend\eval\gemini_refine.py
```

15. assistant (2026-06-20T16:56:37.594Z)

Now let me search for specific output patterns and highlight/reel stitching:

16. user (2026-06-20T16:56:38.857Z)

```
Found 5 files
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\fusion_hybrid.py
backend\pipeline\stitcher\compiler.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\gemini.py
```

17. user (2026-06-20T16:56:38.929Z)

```
1       """One-command rally-segmentation eval ? score a windowing against the golden labels
with the
2       over-segmentation-sensitive metric set, honestly.
3
4       This is the harness the rally-quality project (``docs/RALLY_QUALITY_RESEARCH.md`` §4.5 /
§7)
5       measures every tier against. It closes the loop the over-merge finding exposed: plain
temporal
6       IoU-F1 is blind to merged mega-windows, so we report it **alongside** segmental F1@k,
the
7       segment-count ratio, and the explicit ``merge_split`` breakdown
(``segmentation_metrics``).
8
9       For each golden video it: parses the cached WASB/TrackNet trajectory CSV ? builds
candidate
10      windows under a :class:`~backend.eval.windowing.WindowingPreset` (so it tracks the
SERVED
```

```
    operating point and any future windowing the same way) ? scores the windows-as-rallies against
    the ground-truth rally intervals. Aggregates per-video held-out F1 with a bootstrap CI, and can
    diff two windowings with a paired sign-test + CI (the per-tier ``delta``).

    GROUND TRUTH: the golden manifest (``output/human_lovo_manifest.json``) gives each video's
    trajectory path + fps + frame_width; the GT rally intervals come from
    ``output/<name>_golden_features.json`` (the ``gts`` key) so this needs no extra label files.

    Pure offline (CPU): trajectory parsing + windowing + interval scoring; no GPU/Gemini.
    """
    from __future__ import annotations

    import argparse
    import json
    import os
    from typing import Any, Dict, List, Optional, Tuple

    from backend.eval import eval_stats, metrics, segmentation_metrics, windowing
    from backend.eval.metrics import Interval
    from backend.eval.windowing import PRESETS, SERVED, WindowingPreset

    DEFAULT_MANIFEST = os.path.join("output", "human_lovo_manifest.json")
    DEFAULT_FEATURES_DIR = "output"
    DEFAULT_IOU = 0.5

    #: Long-rally lens threshold (s). A rally is "long" (an eye-catching highlight) when its duration
    #: (end ? start) exceeds this. The local detector over-fires, and its false positives are mostly
    #: SHORT (motion blips, background-court fragments on multicourt footage); filtering to long rallies
    #: strips most of those FPs and reveals the real product quality on the long-rally happy path. Used
    #: as the default split point for the stratified report (``--long-tau`` overrides it). DURATION, not
    #: shot-count, is the filter: golden ``shots_count`` is unlabeled and the no-AI path reports it as
    #: "Unknown", whereas duration is derived from the same start/end labels we already trust.
    LONG_RALLY_TAU = 5.0

    #: Multicourt golden clips ? the court-type lookup for the stratified report. The golden manifest
    #: (``output/human_lovo_manifest.json``) is gitignored, so this committed set is the AUDITABLE source
    #: of the single-vs-multicourt strata, kept in sync with ``docs/GOLDEN_VIDEOS.md`` (the clips tagged
    #: "multicourt"). Multicourt footage carries background games on adjacent courts ? the dominant
    #: source of short false positives ? so we report it separately. A manifest entry MAY carry an
    #: explicit ``court_type`` field, which overrides this set (see :func:`court_type_for`).
    MULTICOURT_CLIPS = frozenset({"mahadevpura_2", "GX010128", "Badminton_BXH_2", "Boxhill_Doubles"})


    def court_type_for(video: Dict[str, Any]) -> str:
        """``'multicourt'`` or ``'single'`` for a golden video. Honors an explicit manifest
        ``court_type`` field when present, else falls back to membership in ``MULTICOURT_CLIPS``
        (keyed by ``name``), else ``'single'``."""
```

```
58          ct = video.get("court_type")
59          if isinstance(ct, str) and ct:
60              return ct
61          return "multicourt" if video.get("name") in MULTICOURT_CLIPS else "single"
62
63
64      def filter_by_min_duration(ivs: List[Interval], min_duration: float) -> List[Interval]:
65          """The long-rally lens: keep only intervals strictly longer than ``min_duration``
seconds.
66
67          ``min_duration <= 0`` is an EXACT passthrough (the default-OFF semantics ? every
real rally has
68          positive duration, so nothing is dropped). It's a FILTER, not a new metric: applied
identically
69          to predictions AND ground truth before any matching, so the tIoU and R2 paths stay
consistent."""
70          if min_duration <= 0.0:
71              return list(ivs)
72          return [iv for iv in ivs if (iv[1] - iv[0]) > min_duration]
73
74
75      # ------------------------------------------------------------------------ #
76      # Pure scoring core (no I/O ? unit-testable)
77      # ------------------------------------------------------------------------ #
78      def score_intervals(preds: List[Interval], gts: List[Interval],
79                          iou: float = DEFAULT_IOU,
80                          tau_start: Optional[float] = None,
81                          tau_end: Optional[float] = None,
82                          min_duration: float = 0.0) -> Dict[str, Any]:
83          """The full per-video metric row for predicted vs ground-truth rally intervals.
84
85          Pairs temporal-IoU P/R/F1 + boundary error (``metrics.score``) with the
86          over-segmentation-sensitive set (F1@k, segment-count ratio, merge/split breakdown)
so a
87          merged mega-window is *visible* even when IoU-F1 isn't moved.
88
89          Also carries the **R2** task-faithful block (``tolerance_metrics``, the headline
going forward;
90          tIoU here is the SECONDARY trend-line per the augment-then-ablation-gate decision):
strict-1:1
91          tolerance-match ``tol_f1/tol_precision/tol_recall`` (over-production costs
precision) + the
92          start/end boundary-error medians + the over-seg guard counts. See
docs/R2_EVAL_METRIC_DESIGN.md.
93
94          ``min_duration`` > 0 applies the **long-rally lens**
(:func:`filter_by_min_duration`): BOTH preds
95          and GTs are filtered to rallies longer than that many seconds BEFORE any matching,
so the whole
96          row (tIoU set AND R2 block) scores only long rallies. Default 0.0 = OFF (exact
passthrough).
97          """
98          from backend.eval import tolerance_metrics as tm
99          preds = filter_by_min_duration(preds, min_duration)
100         gts = filter_by_min_duration(gts, min_duration)
```

18. user (2026-06-20T16:56:39.175Z)

```
1       """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3       Wires the already-tested pieces together:
4         WASB trajectory CSV (Frame,Visibility,X,Y)
5           -> TrackNetRunner.trajectory_to_action_windows(cfg)   [predict_fn]
6           -> backend.eval.calibration (improvement_report / cross_validate)
```

```
7
8       Run (after a full-video WASB pass exists):
9           python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10              --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12      Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13      same GT is an **optimistic** upper bound. The trustworthy number is the
14      cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15      `leave_one_video_out` for precision/F1 generalization).
16      """
17      from __future__ import annotations
18
19      import argparse
20      from typing import List, Tuple, Callable
21
22      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
23      from backend.eval.calibration import (
24          select_best, improvement_report, cross_validate, evaluate,
25      )
26
27      Interval = Tuple[float, float]
28      Config = Tuple[float, float, float]   # (velocity_thresh, merge_gap,
min_window_duration)
29
30      BASELINE: Config = (0.02, 2.0, 2.0)   # the WasbConfig / trajectory_to_action_windows
defaults
31
32
33      def load_golden_gts(csv_path: str) -> List[Interval]:
34          """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
35
36          Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
37          start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
38          driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
39          """
40          from backend.eval.gt_loader import load_gt_intervals
41          return load_gt_intervals(csv_path, strict=False)
42
43
44      def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
45          """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
46          points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
47
48          def predict_fn(cfg: Config) -> List[Interval]:
49              vt, mg, mwd = cfg
50              wins = TrackNetRunner.trajectory_to_action_windows(
51                  points, fps=fps, frame_width=frame_width,
52                  velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
53              )
54              return [(w["start"], w["end"]) for w in wins]
55
56          return predict_fn
57
58
59      def default_grid() -> List[Config]:
60          return [(vt, mg, mwd)
61                  for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
62                  for mg in (1.0, 2.0, 3.0)
63                  for mwd in (1.0, 2.0, 3.0)]
64
65
66      def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
67              iou: float = 0.5) -> dict:
68          gts = load_golden_gts(labels_csv)
```

```
69          if not gts:
70              raise SystemExit(f"no usable golden rallies in {labels_csv}")
71          predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
72          grid = default_grid()
73
74          base = evaluate(predict_fn(BASELINE), gts, iou)
75          best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
76          fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou)   # OPTIMISTIC
(fit on full GT)
77          cv = cross_validate(predict_fn, grid, gts, k=5)                      # HONEST held-
out recall
78
79          print("=== WASB rally-segmentation calibration ===")
80          print(f"golden rallies: {len(gts)}  | IoU thresh: {iou}")
```

19. assistant (2026-06-20T16:56:41.226Z)

Now let me check the stitcher and pipeline for output patterns:

20. user (2026-06-20T16:56:41.910Z)

```
1       import os
2       import subprocess
3       import tempfile
4       import logging
5       from pathlib import Path
6       from typing import Any, Dict, List, Optional, Tuple, Union
7
8       logger = logging.getLogger(__name__)
9
10      # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11      # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12      # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13      # backend/pipeline/stitcher/, so parents[2] == backend/.
14      _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16      class VideoCompiler:
17          def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                       watermark: Optional[Any] = None):
19              self.output_dir = output_dir
20              self.end_padding = end_padding
21              self.begin_padding = begin_padding
22              # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23              # normalized to a dict (or None) so this module stays config-package-agnostic.
24              self.watermark = self._normalize_watermark(watermark)
25              os.makedirs(output_dir, exist_ok=True)
26
27          @staticmethod
28          def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29              if watermark is None:
30                  return None
31              if hasattr(watermark, "model_dump"):
32                  return watermark.model_dump()
33              if isinstance(watermark, dict):
34                  return dict(watermark)
35              return None
36
37          @staticmethod
38          def _ff_quote(path: str) -> str:
```

```
39              """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40              filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41              slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43              The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44              option parser treats ':' as the option separator, so a Windows drive-letter
45              path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46              the encode fails with "Invalid argument". This silently disabled the
47              on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48              return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
49
50          @staticmethod
51          def _parse_time(val: Union[int, float, str]) -> float:
52              """Normalizes a timestamp value to float seconds.
53              Handles: float/int (pass-through), plain numeric strings ("12.4"),
54              and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55              This mirrors the parse_time logic used in gemini.py to ensure
56              the stitcher can always consume whatever format the indexer produced."""
57              if isinstance(val, (int, float)):
58                  return float(val)
59              if isinstance(val, str):
60                  val = val.strip()
61                  if ":" in val:
62                      # Handle MM:SS, HH:MM:SS, etc.
63                      parts = val.split(":")
64                      parts.reverse()
65                      total = 0.0
66                      for i, p in enumerate(parts):
67                          try:
68                              total += float(p) * (60 ** i)
69                          except ValueError:
70                              pass
71                      return total
72                  try:
73                      return float(val)
74                  except ValueError:
75                      logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
76                      return 0.0
77              logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
78              return 0.0
79
80          @staticmethod
81          def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
82              """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
83
84              A highlight reel must never replay the same footage. The same rally can land in
85              the segment list more than once ? e.g. two detector runs accumulated for one
86              video (add_video is an UPSERT, so re-running without a prune leaves both sets),
or
87              chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
rally
88              twice. Merging any range that overlaps or abuts the previous one collapses true
89              duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
90              Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
91              """
92              parsed = []
93              for raw_s, raw_e in segments:
94                  s = VideoCompiler._parse_time(raw_s)
95                  e = VideoCompiler._parse_time(raw_e)
96                  if e > s:
97                      parsed.append((s, e))
98              parsed.sort()
```

```
99              merged: List[Tuple[float, float]] = []
100             for s, e in parsed:
101                 if merged and s <= merged[-1][1]:   # overlaps/abuts previous -> extend it
102                     merged[-1] = (merged[-1][0], max(merged[-1][1], e))
103                 else:
104                     merged.append((s, e))
105             return merged
106
107         def _get_video_duration(self, video_path: str) -> float:
108             """Probes the video file to get its total duration in seconds.
109             Returns 0.0 if it cannot be determined."""
110             try:
111                 result = subprocess.run(
112                     [
113                         "ffprobe", "-v", "error",
114                         "-show_entries", "format=duration",
115                         "-of", "default=noprint_wrappers=1:nokey=1",
116                         video_path
117                     ],
118                     capture_output=True, text=True, check=True
119                 )
120                 return float(result.stdout.strip())
121             except Exception as e:
122                 logger.warning(f"Could not determine video duration via ffprobe: {e}")
123                 return 0.0
124
125         def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
126             """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
127             and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
128             watermarking is disabled or has nothing to draw. The concat stage stays -c copy
129             because the mark is already baked into every clip identically."""
130             wm = self.watermark
131             if not wm or not wm.get("enabled"):
132                 return None
133
134             text = (wm.get("text") or "").strip()
135             logo_path = (wm.get("logo_path") or "").strip()
136             if logo_path and not os.path.exists(logo_path):
137                 logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
overlay.")
138                 logo_path = ""
139             if not text and not logo_path:
140                 return None
141
142             margin = int(wm.get("margin", 24))
143             opacity = float(wm.get("opacity", 0.85))
144             position = str(wm.get("position", "bottom-right"))
145             # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
146             dt_xy = {
147                 "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
148                 "bottom-left": f"x={margin}:y=h-th-{margin}",
149                 "top-right": f"x=w-tw-{margin}:y={margin}",
150                 "top-left": f"x={margin}:y={margin}",
```

21. user (2026-06-20T16:56:42.932Z)

```
6-segment, by diffing tuned-vs-golden quantitatively, or by cutting clips.
7-
8-Reuses (no new windowing\metrics\cutting logic):
9:  - calibrate_wasb.make_predict_fn  -> trajectory -> segments for a given config
10-  - eval.metrics.match_intervals/score -> tuned-vs-golden IoU comparison
11-  - tools.rally_slicer.slice_rallies   -> optional clip cutting (the exported CSV is
12-    written in the golden/slicer schema, so it feeds straight back in)
13-
```

```
14-Run (uses the tuned config from the first calibration by default):
15:    python -m backend.eval.export_wasb_segments --trajectory ~/clips/sample_long_traj.csv \
16:        --labels "Annotation Setup/golden_labels_badminton.csv" --fps 59.94 --frame-width
1920
17-    # add --video <mp4> --slice  to also cut one padded clip per tuned rally
18-"""
19-from __future__ import annotations
--
24-import argparse
25-from typing import List, Tuple, Dict, Optional
26-
27:from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
28-from backend.eval.metrics import match_intervals, score
29-from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30-from backend.pipeline.detectors import rally_gate
--
67-    os.makedirs(osp.dirname(osp.abspath(path)) or ".", exist_ok=True)
68-
69-
70:def write_segments_csv(path: str, segments: List[Interval]) -> str:
71-    """Write segments in the golden schema (rally_number\start_time\end_time + mm:ss).
72-
73-    The schema is what tools.rally_slicer.load_rally_labels consumes, so the file
--
122-    return {"score": sc, "rows": rows}
123-
124-
125:def write_comparison_csv(path: str, comparison: dict) -> str:
126-    _ensure_parent(path)
127-    with open(path, "w", newline="", encoding="utf-8") as f:
128-        w = csv.DictWriter(f, fieldnames=COMP_FIELDS)
--
137-
138-
139-def run(trajectory_csv: str, fps: float, frame_width: float, cfg: Config = TUNED,
140:        out_csv: str = "output/wasb_tuned_segments.csv", labels_csv: Optional[str] = None,
141-        iou: float = 0.5, baseline: bool = False,
142-        comparison_out: Optional[str] = None, min_crossings: int = 0) -> dict:
143-    predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
--
154-        return kept
155-
156-    tuned_segs = _gate(predict_fn(cfg), "tuned")
157:    write_segments_csv(out_csv, tuned_segs)
158-    total = sum(e - s for s, e in tuned_segs)
159-    print("=== WASB tuned-segment export ===")
160-    print(f"tuned cfg {cfg}: {len(tuned_segs)} rally segments, {total:.1f}s of play ->
{out_csv}")
--
163-    if baseline:
164-        base_segs = _gate(predict_fn(BASELINE), "baseline")
165-        base_out = _with_suffix(out_csv, "_baseline")
166:        write_segments_csv(base_out, base_segs)
167-        print(f"baseline cfg {BASELINE}: {len(base_segs)} segments -> {base_out}")
168-        result["num_baseline"] = len(base_segs)
169-        result["baseline_csv"] = base_out
--
175-        comp = build_comparison(tuned_segs, gts, iou)
176-        sc = comp["score"]
177-        comp_out = comparison_out or _with_suffix(out_csv, "_vs_golden")
178:        write_comparison_csv(comp_out, comp)
179-        print(f"\n--- tuned vs golden ({len(gts)} rallies, IoU>={iou}) ---")
180-        print(f"matched (TP) {sc.true_positives} | misses (FN) {sc.false_negatives} | "
181-              f"extra (FP) {sc.false_positives}")
--
```

```
205-    ap.add_argument("--min-duration", type=float, default=TUNED[2])
206-    ap.add_argument("--labels", default=None, help="golden labels CSV -> tuned-vs-golden
comparison")
207-    ap.add_argument("--iou", type=float, default=0.5)
208:    ap.add_argument("--out", default="output/wasb_tuned_segments.csv")
209-    ap.add_argument("--comparison-out", default=None)
210-    ap.add_argument("--baseline", action="store_true", help="also export baseline-config
segments")
211-    ap.add_argument("--min-crossings", type=int, default=0,
```

22. assistant (2026-06-20T16:56:47.778Z)

Now let me search for run logs and other output patterns:

23. user (2026-06-20T16:56:49.160Z)

```
Found 14 files
backend\pipeline\detectors\native_wasb_runner.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\stitcher\compiler.py
backend\pipeline\segmenters\gemini.py
backend\pipeline\detectors\stub_runner.py
backend\pipeline\segmenters\motion.py
backend\pipeline\detectors\tracknet_runner.py
backend\pipeline\detectors\wasb_runner.py
backend\pipeline\detectors\base.py
backend\pipeline\personalization\serving.py
backend\pipeline\detectors\person_detector.py
backend\pipeline\validators\base.py
backend\pipeline\validators\format.py
```

24. user (2026-06-20T16:56:49.281Z)

```
File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-
highlight-indexer. Did you mean config?
```

25. assistant (2026-06-20T16:56:51.521Z)

Let me find the correct config path:

26. user (2026-06-20T16:56:51.994Z)

```
No files found
```

27. user (2026-06-20T16:56:52.361Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer/config.json
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_config.py
C:\Users\avidu\Projects\badminton-highlight-indexer/tests/test_config_wasb.py
```

28. assistant (2026-06-20T16:56:54.204Z)

Let me read the config files and search for pipeline initialization:

29. user (2026-06-20T16:56:54.694Z)

```
1       {
2           "sport": "badminton",
3           "ai_provider": "gemini",
4           "ai_model": "gemini-flash-latest",
5           "validation": {
6               "min_resolution_width": 1280,
```

```
            "min_resolution_height": 720,
            "min_fps": 29.9,
            "min_blur_threshold": 20.0,
            "max_scene_cuts_per_minute": 0.5,
            "relevance": {
                "court_green_lower": [
                    35,
                    40,
                    40
                ],
                "court_green_upper": [
                    85,
                    255,
                    255
                ],
                "court_pixels_min_ratio": 0.05
            }
        },
        "indexing": {
            "default_segmenter": "gemini",
            "use_gpu": true,
            "motion_threshold": 0.08,
            "min_segment_duration": 3.0,
            "dead_time_merge_gap": 2.0,
            "chunk_duration": 90,
            "chunk_overlap": 10,
            "detector_model": "fasterrcnn_resnet50_fpn",
            "detector_score_threshold": 0.5,
            "yolo_velocity_threshold": 0.15,
            "yolo_frame_skip": 3,
            "yolo_tracker": "bytetrack.yaml",
            "yolo_prefetch_workers": 2,
            "min_action_window_duration": 2.0,
            "log_yolo_candidates": true,
            "store_yolo_candidates": true,
            "ai_handoff_padding": 2.0,
            "ai_max_workers": 1,
            "tracknet": {
                "wsl_distro": "Ubuntu",
                "repo_dir": "~/models/TrackNetV4",
                "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
                "conda_env": "TrackNetV4",
                "weights_path": "",
                "queue_length": 5,
                "stage_in_wsl": true,
                "wsl_stage_dir": "~/clips",
                "timeout_sec": 1800,
                "velocity_threshold": 0.02
            },
            "wasb": {
                "wsl_distro": "Ubuntu",
                "repo_dir": "~/models/WASB-SBDT",
                "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
                "conda_env": "wasb",
                "weights_path": "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
                "sport": "badminton",
                "wsl_stage_dir": "~/clips",
                "timeout_sec": 1800,
                "velocity_threshold": 0.02
            }
        },
        "output": {
            "default_highlights_name": "highlights.mp4",
            "db_name": "sports_indexer.db"
```

```
        },
        "stitching": {
            "begin_padding": 0.5,
            "end_padding": 0.5,
            "use_cache": false,
            "min_shot_count": 1,
            "watermark": {
                "enabled": true,
                "text": "Generated by Khelsutra.guru",
                "position": "bottom-right",
                "opacity": 0.85,
                "logo_path": "",
                "font_path": ""
            }
        }
    }
```

30. user (2026-06-20T16:56:55.204Z)

```
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/annotations/automated_provider.py:
No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/annotations/base.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/annotations/file_provider.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/annotations/__init__.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/api/job_worker.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/api/uploads.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/api/__init__.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/config/loader.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/config/models.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/config/__init__.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/ablation.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/ablation_pdf.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/annotations.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/audio/e1_probe.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/audio/__init__.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/batch.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/calibrate_local.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/calibrate_wasb.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/calibration.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/classifier.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/distill_local.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/eval_partial_labels.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/eval_stats.py: No such file or
directory
```

```
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/experiment.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/export_wasb_segments.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/fusion_audio.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/fusion_compare.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/fusion_golden.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/gemini_refine.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/golden_fixtures.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/golden_real_fixtures.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/gt_loader.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/harness.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/metrics.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/per_user_eval.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/per_user_gate.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/per_user_regression.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/promotion.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/rally_seg_eval.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/rally_seq_proto.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/regression.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/score_calibration.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/segmentation_metrics.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/served_gate_a.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/serve_contrast.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/splits.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/tolerance_metrics.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/training.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/windowing.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/eval/__init__.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/features/rally_seq.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/features/__init__.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/infrastructure/database.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/infrastructure/execution_policy.py:
No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/main.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/audio/hit_detector.py: No
such file or directory
```

```
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/audio/__init__.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/detectors/audio_features.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/detectors/base.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/detectors/court_gate.py:
No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/detectors/fusion_features.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/detectors/native_wasb_runner.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/detectors/person_detector.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/detectors/rally_gate.py:
No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/detectors/stub_runner.py:
No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/detectors/tracknet_runner.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/detectors/wasb_infer.py:
No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/detectors/wasb_runner.py:
No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/detectors/__init__.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/personalization/serving.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/personalization/__init__.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/segmenters/base.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/segmenters/fusion_hybrid.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/segmenters/gemini.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/segmenters/motion.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/segmenters/tracknet_hybrid.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/segmenters/trajectory_hybrid.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/segmenters/wasb_hybrid.py:
No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/segmenters/yolo_hybrid.py:
No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/segmenters/_keyhint.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/segmenters/__init__.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/stitcher/compiler.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/validators/base.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/validators/blur.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/validators/format.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/validators/pts_continuity.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/validators/relevance.py:
No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-
indexerbackend/pipeline/validators/resolution_fps.py: No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/validators/scene_cut.py:
```

```
No such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/pipeline/validators/__init__.py: No
such file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/providers/base.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/providers/gemini.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/providers/__init__.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/reporting/harvest.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/reporting/run_signals.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/reporting/__init__.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/results.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/sports/badminton.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/sports/base.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/sports/__init__.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/storage/base.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/storage/gcs.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/storage/gdrive.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/storage/local.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/storage/ref.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/storage/s3.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/storage/__init__.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/tools/export.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/tools/rally_slicer.py: No such file
or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/tools/__init__.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/utils/geometry.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/utils/hashing.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/utils/logging_setup.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/utils/paths.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/utils/run_telemetry.py: No such
file or directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/utils/video.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/utils/__init__.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/worker/__init__.py: No such file or
directory
grep: C:UsersaviduProjectsbadminton-highlight-indexerbackend/worker/__main__.py: No such file or
directory
```

31. assistant (2026-06-20T16:56:56.780Z)

Let me use PowerShell to search properly:

32. user (2026-06-20T16:56:58.448Z)

```
(PowerShell completed with no output)
```

33. user (2026-06-20T16:56:58.457Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      import logging
13      from pathlib import Path
14      from typing import Any
15
16      from pydantic import BaseModel
17
18      from .models import AppConfig
19
20      logger = logging.getLogger(__name__)
21
22      DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25      def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
"") -> list[str]:
26          """Dotted paths in `data` that the typed config will not honor.
27
28          Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
29          kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
30          typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
31          user's intent is lost ? collect every such key so load_config can warn.
32          """
33          unknown: list[str] = []
34          fields = model_cls.model_fields
35          for key, val in data.items():
36              if key not in fields:
37                  unknown.append(prefix + key)
38                  continue
39              ann = fields[key].annotation
40              if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
41                  unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}."))
42          return unknown
43
44
45      def _get_builtin_defaults() -> dict[str, Any]:
46          """Return a plain dict of the current built-in defaults.
47
48          The AppConfig model is the single source of truth for defaults.
49          """
50          return AppConfig().model_dump(mode="json")
51
52
53      def load_config(path: str | Path | None = None) -> AppConfig:
54          """Load configuration with the following precedence:
55
56          1. Built-in defaults defined in the Pydantic models.
57          2. Values from the JSON file (if present and readable).
```

```
        3. (Future) environment / CLI overrides.

        Missing file or unreadable file ? returns a fully defaulted AppConfig.
        Unknown keys in the file are tolerated (extra="allow" on the model).
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

        file_data: dict[str, Any] = {}
        if cfg_path.exists():
            try:
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")

        # A config.json that is valid JSON but not an OBJECT (e.g. `[]`, `42`, `"x"`,
`null`) would
        # otherwise crash startup: a truthy non-mapping hits `.items()` in
_unknown_key_paths, and any
        # non-mapping breaks the `{**defaults, **file_data}` unpack. Degrade to defaults +
warn, per the
        # loader's "bad input is non-fatal" contract.
        if not isinstance(file_data, dict):
            logger.warning("%s is not a JSON object (got %s); ignoring it and using
defaults.",
                           cfg_path, type(file_data).__name__)
            file_data = {}

        if file_data:
            unknown = _unknown_key_paths(file_data, AppConfig)
            if unknown:
                logger.warning(
                    "[CONFIG WARNING] %s contains keys the typed config does not "
                    "recognize (typo'd or removed knobs ? top-level extras are kept "
                    "but unread; unknown section keys are silently dropped): %s",
                    cfg_path, ", ".join(sorted(unknown)),
                )

        # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
        merged = {**_get_builtin_defaults(), **file_data}

        return AppConfig.model_validate(merged)


    def save_default_config(path: str | Path | None = None) -> Path:
        """Write a fresh default config.json.

        Used by scripts/doctor.py diagnostics and the --setup CLI path.
        Overwrites if the file exists.
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
        cfg = AppConfig()
        cfg_path.parent.mkdir(parents=True, exist_ok=True)
        cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
        return cfg_path


    # Backwards-compatible alias used by some older call sites during transition.
    def get_default_config() -> dict[str, Any]:
        """Return defaults as a plain dict (for transitional use only)."""
        return _get_builtin_defaults()

```

34. user (2026-06-20T16:56:58.458Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
43          velocity_threshold: float = 0.02
44          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
45          # decoded frame cache) are deleted automatically ? they are pure intermediates,
46          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
47          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
48          keep_frames: bool = False
49          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
50          # 0 = disabled.
51          min_free_gb: float = 0.0
52
53
54      class WasbIndexConfig(BaseModel):
55          """Typed config for the live WASB shuttle substrate (indexing.wasb).
56
57          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
58          these knobs actually take effect ? previously this whole block was silently dropped
59          because nested config sections defaulted to extra='ignore'.
60          """
61          wsl_distro: str = "Ubuntu"
62          repo_dir: str = "~/models/WASB-SBDT"
```

```
63          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
64          conda_env: str = "wasb"
65          weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
66          sport: str = "badminton"
67          wsl_stage_dir: str = "~/clips"
68          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
69          velocity_threshold: float = 0.02
70          # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
71          # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
72          device: str = "cuda"        # torch device for the native runner: cuda | mps | cpu
73          python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no
`conda activate`)
74          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
75          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
76          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
77          keep_frames: bool = False
78          # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 =
disabled.
79          min_free_gb: float = 0.0
80          # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
81          # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
82          # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
83          # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
84          # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
85          # Tri-state:
86          #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
87          #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
88          #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
89          stream_video: Optional[bool] = None
90
91
92      class FusionConfig(BaseModel):
93          """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
94
95          Every default reproduces control behaviour: the fusion segmenter persists the
96          shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
97          true ? the person-cue features, but it does NOT gate the served handoff unless a
98          threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
99          is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
100         supplied ? off-court persons (spectators / adjacent courts) are excluded.
101         """
102         # Which trajectory substrate seeds the windows (shuttle stays primary).
103         substrate: Literal["wasb", "tracknet"] = "wasb"
104         # Windowing velocity threshold for the fusion family (the engine reads
105         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
106         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
107         velocity_threshold: float = 0.02
108         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
109         # enabled ? so the default path never imports torch / touches the GPU.
110         cue_flags: dict[str, bool] = Field(default_factory=dict)
```

```
111         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
112         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
113         min_crossings: int = Field(default=0, ge=0)
114         # Served Gate B: when the person cue is on, drop windows with median in-court
players
115         # < this. 0 = OFF.
116         min_players: int = Field(default=0, ge=0)
117         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
118         court_polygon: Optional[list[list[float]]] = None
119         # Net line for the person two-sided-motion split (person features only ? the shuttle
120         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
121         net_axis: Literal["x", "y"] = "y"
122         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
123
124
125     class IndexingConfig(BaseModel):
126         default_segmenter: str = "yolo_hybrid"
127         use_gpu: bool = True
128         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
129         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
130         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
131         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
132         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
133         detector_impl: str = "auto"
134         motion_threshold: float = 0.08
135         min_segment_duration: float = 3.0
136         dead_time_merge_gap: float = 2.0
137         chunk_duration: float = 90.0
138         chunk_overlap: float = 10.0
139         detector_model: str = "fasterrcnn_resnet50_fpn"
140         detector_score_threshold: float = 0.5
141         yolo_velocity_threshold: float = 0.15
142         yolo_frame_skip: int = 3
143         yolo_tracker: str = "bytetrack.yaml"
144         yolo_prefetch_workers: int = 2
145         min_action_window_duration: float = 2.0
146         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
147         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
148         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
149         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
150         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
151         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
152         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
153         max_window_duration: float = 6.0
154         window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
point
155         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
156         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
157         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
158         # Only cuts (recall can't drop). OFF by default until measured/promoted.
159         window_hard_cap: bool = False
160         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
```

```
slowly
161         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
162         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
163         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
164         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
165         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
166         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
167         window_inactivity_end: float = 1.5
168         inactivity_rally_thresh: float = 0.20
169         inactivity_min_density: float = 0.30
170         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after
171         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
172         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
173         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
174         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
175         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
176         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
177         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
178         window_blob_fallback: bool = False
179         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
180         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
181         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
182         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
183         window_blob_factor: float = 4.0
184         log_yolo_candidates: bool = True
185         store_yolo_candidates: bool = True
186         ai_handoff_padding: float = 2.0
187         ai_max_workers: int = 5
188         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
189         # chunks of a high-bitrate (4K) source blow past the provider upload cap
190         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
191         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
192         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
193         # Matches gemini_refine's --clip-scale-width default.
194         ai_chunk_scale_width: int = 1280
195         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
196         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
197         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
198         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
199         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
200         skip_ai_handoff: bool = False
201         # Optional debug knob: trim every video to the first N seconds before processing.
202         debug_duration: Optional[float] = None
203
```

```
        tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
        wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
        fusion: FusionConfig = Field(default_factory=FusionConfig)

        @field_validator("default_segmenter")
        @classmethod
        def segmenter_must_be_registered(cls, v: str) -> str:
            # Registry check happens at runtime in main/segmenter selection.
            # Here we just allow any string for forward compatibility.
            return v

        @model_validator(mode="after")
        def _chunk_step_must_be_positive(self) -> "IndexingConfig":
            # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
            # step makes `while t < duration: t += step` loop forever, growing memory before
any
            # GPU work. Reject the bad config at load time instead.
            if self.chunk_overlap >= self.chunk_duration:
                raise ValueError(
                    f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
                    f"({self.chunk_duration}); otherwise chunking never advances."
                )
            return self


    class OutputConfig(BaseModel):
        default_highlights_name: str = "highlights.mp4"
        db_name: str = "sports_indexer.db"
        # Directory where the DB, highlight reels, and processed clips are written.
        # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
        output_dir: str = "output"


    class WatermarkConfig(BaseModel):
        """Branding overlay burned into each highlight clip during the per-clip re-encode
        (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
        (under `stitching.watermark` in config.json) to turn it off. The text is fully
        configurable. The concat stage stays stream-copy (-c copy)."""
        enabled: bool = True
        text: str = "Generated by Khelsutra.guru"
        logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
        font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family
below
        font: str = "Sans"         # fontconfig family used when font_path is empty
        font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a
fraction of frame height
        opacity: float = Field(default=0.85, ge=0.0, le=1.0)
        box: bool = True           # faint backing box behind the text for legibility
        margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
        position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-
right"


    class StitchingConfig(BaseModel):
        begin_padding: float = 0.5
        end_padding: float = 0.5
        use_cache: bool = False
        min_shot_count: int = 1
        # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
        # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
        # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
```

```
261         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
262         # local detector over-fires and its false positives are mostly SHORT (motion blips,
263         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
264         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
265         min_rally_duration: float = Field(default=0.0, ge=0.0)
266         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
267
268
269     class LoggingConfig(BaseModel):
270         """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
271         # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
272         # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
273         # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
274         # them to WARNING; set true to restore full chatter for request-flow debugging.
275         verbose_http: bool = False
276
277
278     class SecurityConfig(BaseModel):
279         # When set, /api/process video_path must resolve under this directory (hosted/tenant
280         # mode). Default None preserves the single-user local 'any local file' workflow.
281         allowed_video_root: Optional[str] = None
282
283         # --- Chunked upload (B2, PR-2) ---
284         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
285         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
286         # contain upload_dir or /api/process will reject the uploaded paths.
287         upload_dir: str = "output/uploads"
288         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
289         # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
290         max_upload_mb: int = 4096
291         max_part_mb: int = 95
292         allowed_upload_extensions: List[str] = ["mp4", "mov"]
293
294
295     class StorageConfig(BaseModel):
296         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
297         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
298         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
299         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
300         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
301         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
302         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
303         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
304         output_root: str = ""
305         local: dict[str, Any] = Field(default_factory=dict)
306         s3: dict[str, Any] = Field(default_factory=dict)
307         gcs: dict[str, Any] = Field(default_factory=dict)
308         gdrive: dict[str, Any] = Field(default_factory=dict)
309
310
311     class AppConfig(BaseModel):
312         """Root configuration object.
313
314         All runtime configuration should be accessed via an instance of this class
315         (or the loader) rather than raw dict lookups.
316         """
317
```

```
        sport: Sport = "badminton"
        ai_provider: str = "gemini"
        # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
        # the alias never deprecates underneath us. Eval reports record the run date
alongside.
        ai_model: str = "gemini-flash-latest"

        validation: ValidationConfig = Field(default_factory=ValidationConfig)
        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
        output: OutputConfig = Field(default_factory=OutputConfig)
        stitching: StitchingConfig = Field(default_factory=StitchingConfig)
        security: SecurityConfig = Field(default_factory=SecurityConfig)
        logging: LoggingConfig = Field(default_factory=LoggingConfig)
        storage: StorageConfig = Field(default_factory=StorageConfig)

        # Allow extra keys during transition so existing config.json files do not break.
        model_config = {
            "extra": "allow",
            "validate_assignment": True,
        }

        # Temporary migration shim for legacy dict-style access (config.get("section",
{}).get(...)).
        # Will be removed once all call sites (validators, segmenters, etc.) move to typed
        # attribute access / the typed config. A nested *section* is returned as a plain
dict
        # via model_dump, so legacy chained access keeps working ? returning the Pydantic
        # section model instead breaks it (a model has no .get), which bit every validator
on
        # real runs. Scalar leaves are returned as-is.
        def get(self, key: str, default: Any = None) -> Any:
            _MISSING = object()
            val: Any = _MISSING
            if key in type(self).model_fields or hasattr(self, key):
                val = getattr(self, key, _MISSING)
            if val is _MISSING:
                # Legacy convenience: find a leaf key inside any nested section. Sections
are
                # discovered dynamically from the model fields, so a future top-level
section
                # (e.g. `audio`, `jobs`) is covered automatically without editing a
hardcoded list.
                for fname in type(self).model_fields:
                    section = getattr(self, fname, None)
                    if isinstance(section, BaseModel) and hasattr(section, key):
                        val = getattr(section, key)
                        break
            if val is _MISSING:
                return default
            if isinstance(val, BaseModel):
                return val.model_dump(mode="json")
            return val

        def __getitem__(self, key: str) -> Any:
            val = self.get(key)
            if val is None and key not in type(self).model_fields:
                # Re-raise like a real dict for unknown top-level
                raise KeyError(key)
            return val


    # Convenience type alias for callers that want to annotate
    Config = AppConfig

```

35. assistant (2026-06-20T16:57:00.756Z)

Now let me search for specific write locations and compile operations:

36. user (2026-06-20T16:57:01.530Z)

```
150                 "top-left": f"x={margin}:y={margin}",
151             }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
152             ov_xy = {
153                 "bottom-right": f"W-w-{margin}:H-h-{margin}",
154                 "bottom-left": f"{margin}:H-h-{margin}",
155                 "top-right": f"W-w-{margin}:{margin}",
156                 "top-left": f"{margin}:{margin}",
157             }.get(position, f"W-w-{margin}:H-h-{margin}")
158 
159             drawtext = ""
160             if text:
161                 ratio = float(wm.get("font_size_ratio") or 0.035)
162                 divisor = max(1, round(1.0 / ratio))
163                 # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
164                 tf_path = os.path.join(tmp_dir, "watermark.txt")
165                 with open(tf_path, "w", encoding="utf-8") as fh:
166                     fh.write(text)
167                 # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
168                 # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
169                 font_path = (wm.get("font_path") or "").strip()
170                 if not font_path and _BUNDLED_FONT.exists():
171                     font_path = str(_BUNDLED_FONT)
172                 if font_path:
173                     font_opt = f"fontfile='{self._ff_quote(font_path)}'"
174                 else:
175                     font_opt = f"font='{wm.get('font', 'Sans')}'"
176                 box = ""
177                 if wm.get("box", True):
178                     box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
179                 drawtext = (
180                     f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
181                     f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
182                 )
183 
184             if logo_path and drawtext:
185                 return
f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
186             if logo_path:
187                 return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
188             return drawtext
189 
190         def _trim_one_segment(
191             self,
192             idx: int,
193             raw_start: Union[int, float, str],
194             raw_end: Union[int, float, str],
195             segments: List[Tuple[float, float]],
196             video_duration: float,
197             tmp_dir: str,
198             raw_video_path: str,
199             watermark_filter: Optional[str],
200         ) -> Tuple[Optional[str], Optional[Tuple[int, float, float, str]]]:
201             """Trim a single segment into its own clip file (extracted verbatim from the
202             per-segment body of compile_highlights). Returns (clip_path, skip_record): on
203             success the clip path and None; on a skip (invalid/out-of-range/ffmpeg failure)
204             None and the (idx, start, end, reason) record to record in skipped_segments."""
205             # Normalize timestamps to float seconds ? handles float, int,
```

```
            # plain numeric strings, and colon-separated formats like "MM:SS"
            start = self._parse_time(raw_start)
            end = self._parse_time(raw_end)
            segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s - {end:.2f}s]"

            # Validate the segment times
            if start < 0:
                logger.warning(f"{segment_label}: start_time is negative ({start:.2f}s).
Clamping to 0.")
                start = 0.0

            if start >= end:
                logger.error(f"{segment_label}: start_time ({start:.2f}s) >= end_time
({end:.2f}s). Skipping invalid segment.")
                return None, (idx, start, end, "start >= end")

            # Check if segment extends beyond video duration
            if video_duration > 0:
                if start >= video_duration:
                    logger.error(f"{segment_label}: start_time ({start:.2f}s) is beyond the
video duration ({video_duration:.2f}s). "
                                 f"This segment cannot be extracted ? the video ran out.
Skipping.")
                    return None, (idx, start, end, "start beyond video end")

                if end > video_duration:
                    original_end = end
                    end = video_duration
                    logger.warning(f"{segment_label}: end_time ({original_end:.2f}s) exceeds
video duration ({video_duration:.2f}s). "
                                   f"Clamping end_time to {video_duration:.2f}s. The segment
may be truncated.")

            clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")

            # Precise sub-second trim using fast re-encoding to preserve stream headers
            padded_start = max(0.0, start - self.begin_padding)
            padded_end = end + self.end_padding

            # Clamp padded_end to video duration if known
            if video_duration > 0 and padded_end > video_duration:
                padded_end = video_duration
                logger.info(f"{segment_label}: Padded end ({end + self.end_padding:.2f}s)
exceeds video duration. "
                            f"Clamping to {video_duration:.2f}s (end padding partially
applied).")

            trim_duration = padded_end - padded_start
            logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")

            base_cmd = [
                "ffmpeg", "-y",
                "-ss", str(round(padded_start, 3)),
                "-to", str(round(padded_end, 3)),
                "-i", raw_video_path,
                "-c:v", "libx264",
                "-preset", "superfast",
                "-crf", "22",
                "-c:a", "aac",
                "-b:a", "128k",
            ]
            vf_args = ["-vf", watermark_filter] if watermark_filter else []

            # Try with the watermark first; if that encode fails (e.g. a bad font/logo
```

```
            # path), retry once WITHOUT it so a branding misconfig can never nuke the
            # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
            attempts = [vf_args, []] if vf_args else [[]]
            produced = False
            last_exc: Optional[subprocess.CalledProcessError] = None
            cmd = base_cmd + [clip_path]
            for attempt_vf in attempts:
                cmd = base_cmd + attempt_vf + [clip_path]
                try:
                    subprocess.run(cmd, capture_output=True, check=True)
                except subprocess.CalledProcessError as e:
                    last_exc = e
                    if attempt_vf and vf_args:
                        _err = (e.stderr.decode(errors="replace").strip()[-300:]
                                if getattr(e, "stderr", None) else "")
                        logger.warning(
                            f"{segment_label}: watermark encode failed; retrying without it.
"
                            f"ffmpeg: {_err or '(no stderr captured)'}"
                        )
                        continue
                    break
                if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
                    produced = True
                    if not attempt_vf and vf_args:
                        logger.warning(f"{segment_label}: encoded WITHOUT the watermark (the
watermark filter failed).")
                    break
                last_exc = None  # ffmpeg succeeded but produced nothing
                break

            if produced:
                return clip_path, None
            elif last_exc is not None:
                stderr_msg = last_exc.stderr.decode(errors='replace').strip()
                logger.error(f"{segment_label}: FFmpeg trim failed.")
                print(f"  Command: {' '.join(cmd)}")
                print(f"  FFmpeg stderr: {stderr_msg[-500:]}")  # Last 500 chars to avoid
flooding
                return None, (idx, start, end, f"ffmpeg error: {stderr_msg[-200:]}")
            else:
                logger.error(f"{segment_label}: FFmpeg exited successfully but output clip
is missing or empty. Skipping.")
                return None, (idx, start, end, "empty output clip")

        def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
            """
            Extracts selected segments from a raw video and stitches them together
            into a seamless highlights file, applying end_padding to prevent abrupt endings.
            """
            if not segments:
                raise ValueError("No highlight segments provided to compile.")

            # Never stitch the same footage twice: collapse overlapping/duplicate ranges
            # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
            original_count = len(segments)
            segments = self._merge_overlapping(segments)
            if len(segments) < original_count:
                logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
-> %d.",
                            original_count - len(segments), original_count, len(segments))
            if not segments:
                raise ValueError("No valid (start < end) highlight segments after merge.")

```

```
321             if not os.path.exists(raw_video_path):
322                 raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
323
324             # Defense-in-depth: never let the output name escape output_dir (the API also
325             # validates this at the request boundary).
326             output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
327
328             # Probe video duration so we can detect out-of-range segments
329             video_duration = self._get_video_duration(raw_video_path)
330             if video_duration > 0:
331                 logger.info(f"Source video duration: {video_duration:.2f}s")
332             else:
333                 logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
334
335             # We will create temporary clips and stitch them using FFmpeg concat demuxer
336             temp_clips = []
337             skipped_segments = []
338
339             # Create a temp directory within output_dir
340             with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
341                 logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
342                 # Build the (optional) watermark filtergraph once; it is applied during
every
343                 # per-clip re-encode so the mark is baked into each clip identically.
344                 watermark_filter = self._watermark_filter(tmp_dir)
345                 if watermark_filter:
346                     logger.info("[watermark] applying branding overlay to each clip.")
347                 for idx, (raw_start, raw_end) in enumerate(segments):
348                     clip_path, skip_record = self._trim_one_segment(
349                         idx, raw_start, raw_end, segments, video_duration,
```

37. user (2026-06-20T16:57:02.283Z)

```
15-
16-class VideoCompiler:
17:    def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float = 0.5,
18-                 watermark: Optional[Any] = None):
19:        self.output_dir = output_dir
20-        self.end_padding = end_padding
21-        self.begin_padding = begin_padding
--
23-        # normalized to a dict (or None) so this module stays config-package-agnostic.
24-        self.watermark = self._normalize_watermark(watermark)
25:        os.makedirs(output_dir, exist_ok=True)
26-
27-    @staticmethod
--
164-            tf_path = os.path.join(tmp_dir, "watermark.txt")
165-            with open(tf_path, "w", encoding="utf-8") as fh:
166:                fh.write(text)
167-            # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
168-            # without fontconfig) ? fontconfig family (last-resort, environment-dependent).
--
231-                                f"Clamping end_time to {video_duration:.2f}s. The segment may
be truncated.")
232-
233:        clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
234-
235-        # Precise sub-second trim using fast re-encoding to preserve stream headers
--
301-            return None, (idx, start, end, "empty output clip")
302-
```

```
303:    def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float, float]],
output_filename: str) -> str:
304-        """
305-        Extracts selected segments from a raw video and stitches them together
--
322-            raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
323-
324:        # Defense-in-depth: never let the output name escape output_dir (the API also
325-        # validates this at the request boundary).
326:        output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
327-
328-        # Probe video duration so we can detect out-of-range segments
--
337-        skipped_segments = []
338-
339:        # Create a temp directory within output_dir
340:        with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
341-            logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
342-            # Build the (optional) watermark filtergraph once; it is applied during every
--
380-                    # Escape single quotes in path
381-                    safe_path = clip.replace("'", "'\\''")
382:                    f.write(f"file '{safe_path}'\n")
383-
384-            logger.info("Stitching clips together...")
--
404-
405-                output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
406:                logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
407-                return output_path
408-            except subprocess.CalledProcessError as e:
```

38. assistant (2026-06-20T16:57:09.146Z)

Perfect. Now let me search for job results and run logs:

39. user (2026-06-20T16:57:10.375Z)

```
Found 3 files
backend\api\job_worker.py
backend\api\static\app.js
backend\api\static\style.css
```

40. user (2026-06-20T16:57:10.546Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api/job_worker.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api/uploads.py
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\api/__init__.py
```

41. user (2026-06-20T16:57:13.069Z)

```
1       """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3       Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4       The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5       that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6       re-claimed when due ? with no central scheduler (the DB is the clock).
7
8       The monkeypatch contract these functions rely on (and that tests depend on):
9       every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
```

```
    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
    `backend.main` module object at call-time (`import backend.main as _main`), never as a
    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
    ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
    inside the functions also avoids the circular import at module load (main.py imports
this
    module; this module must not import main at import time).
    """
    import logging
    import threading
    import traceback
    from concurrent.futures import ThreadPoolExecutor
    from typing import Any, Dict, Optional

    logger = logging.getLogger(__name__)

    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
    # already parallelize their AI handoff internally via their own pools.
    _job_executor: Optional[ThreadPoolExecutor] = None


    def _get_executor() -> ThreadPoolExecutor:
        """Lazy module-global executor; tests monkeypatch this for inline execution."""
        global _job_executor
        if _job_executor is None:
            _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
        return _job_executor


    class JobWaiting(Exception):
        """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
        WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
        NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
        releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
        is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.

        The wiring is additive: the production segmenter path never raises this today (zero
        behaviour change), so a job runs exactly as before. The hook is what a later slice
        (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
        """

        def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
            super().__init__(f"job parked for {delay_sec:.0f}s")
            self.delay_sec = max(0.0, float(delay_sec))
            self.progress = progress


    def _job_to_request(job: Dict[str, Any]):
        """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
        re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
        the original submit. The row stores exactly the ProcessRequest fields."""
        import backend.main as _main
        return _main.ProcessRequest(
            video_path=job["video_path"],
            player_pool=job.get("player_pool"),
            max_frames=job.get("max_frames"),
            segmenter=job.get("segmenter"),
```

```
69          )
70
71
72      def _run_claimed_job(job: Dict[str, Any]) -> None:
73          """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
74          terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
75
76          Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
77          the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
78          result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
79          job self-scheduled and will be re-claimed when `next_poll_at` is due.
80          """
81          import backend.main as _main
82          db_instance = _main.db_instance
83          job_id = job["id"]
84          req = _main._job_to_request(job)
85          try:
86              result = _main._execute_processing(
87                  req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
88              if result.get("video_id"):
89                  db_instance.set_job_video_id(job_id, result["video_id"])
90              db_instance.mark_job_done(job_id, result)
91          except _main.JobWaiting as w:
92              # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
93              logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
94              db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
95          except Exception as e:
96              logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
97              db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
98
99
100     def _drain_due_jobs() -> int:
101         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
102         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
103
104         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
105         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
106         a single-worker box keeps making progress with no central timer (the DB is the
clock).
107         Returns the number of jobs that reached a terminal/parked state this tick.
108         """
109         import backend.main as _main
110         ran = 0
111         while True:
112             job = _main.db_instance.claim_due_job()
113             if job is None:
114                 break
115             ran += 1
116             _main._run_claimed_job(job)
117         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
118         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
119         # on the next submit/drain. Best-effort ? never raises into the worker.
120         _main._schedule_next_drain_if_waiting()
121         return ran
122
123
124     def _schedule_next_drain_if_waiting() -> None:
```

```
125         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
126         self-scheduled job resumes with no further client submit. The drain itself re-checks
127         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
128         import backend.main as _main
129         try:
130             delay = _main.db_instance.seconds_until_next_due()
131         except Exception as e:  # pragma: no cover - defensive; never wedge the worker
132             logger.error(f"Could not compute next-due delay: {e}")
133             return
134         if delay is None:
135             return  # nothing waiting
136         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
137         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)))
138
139
140     def _arm_wake_timer(delay_sec: float) -> None:
141         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
142         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
143         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
144         import backend.main as _main
145         timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
146         timer.daemon = True
147         timer.start()
148
149
150     #: A parked job further out than this is re-checked by a capped wake rather than one
long
151     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
152     _MAX_DRAIN_WAKE_SEC = 60.0
153
```

42. user (2026-06-20T16:57:13.350Z)

```
1       import os
2       import sys
3       import argparse
4       import logging
5       import uuid
6       import uvicorn
7       from dotenv import load_dotenv
8
9       load_dotenv()
10
11      # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
12      logging.basicConfig(
13          level=logging.INFO,
14          format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
15          datefmt='%H:%M:%S'
16      )
17      logger = logging.getLogger(__name__)
18      from fastapi import FastAPI, HTTPException
19      from fastapi.middleware.cors import CORSMiddleware
20      from pydantic import BaseModel
21      from typing import List, Dict, Any, Optional
22
23      from fastapi.staticfiles import StaticFiles
24      from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
25      from backend.pipeline.validators.base import registry
```

```
26      from backend.infrastructure.database import Database
27      from backend.pipeline.segmenters.base import segmenter_registry
28      from backend.pipeline.stitcher.compiler import VideoCompiler
29      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
30      from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
31      from backend.config import load_config as load_app_config, AppConfig, StitchingConfig  #
new typed config
32      from backend.results import Failure  # standardized error/result contract
33      from backend.reporting import emit_indexer_report
34
35      app = FastAPI(title="Sports Highlight Indexer API")
36
37      # Enable CORS for the local web interface. allow_origins='*' with allow_credentials=True
is
38      # rejected by browsers per the fetch spec and is an unsafe default to carry into the
39      # auth/tenancy work; this tool uses no cookies, so credentials stay off.
40      app.add_middleware(
41          CORSMiddleware,
42          allow_origins=["*"],
43          allow_credentials=False,
44          allow_methods=["*"],
45          allow_headers=["*"],
46      )
47
48      # Serves static frontend files directly (now under api/ per approved structure)
49      os.makedirs("backend/api/static", exist_ok=True)
50      app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
51
52      @app.get("/", response_class=HTMLResponse)
53      def serve_index():
54          index_path = "backend/api/static/index.html"
55          if os.path.exists(index_path):
56              with open(index_path, "r", encoding="utf-8") as f:
57                  return f.read()
58          return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
59
60      # Global variables initialized at startup
61      db_instance: Optional[Database] = None
62      global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
63
64      # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
65      # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
66      # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
67      # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
68      # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
69      # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
70      # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
71      from backend.api.job_worker import (  # noqa: F401  (re-exports: names stay addressable
on backend.main)
72          JobWaiting,
73          _arm_wake_timer,
74          _drain_due_jobs,
75          _get_executor,
76          _job_to_request,
77          _MAX_DRAIN_WAKE_SEC,
78          _run_claimed_job,
79          _schedule_next_drain_if_waiting,
80      )
81
82      # Legacy thin wrapper for any remaining direct calls during transition.
83      # New code should import load_app_config / AppConfig from backend.config directly.
```

```
    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        cfg = load_app_config(config_path)
        return cfg.model_dump(mode="json")

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

    class CompileRequest(BaseModel):
        video_id: str
        segment_ids: List[int]
        output_filename: str

    def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
        """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

        On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
        fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
        keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
        """
        if segments:
            db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
        else:
            db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)


    # --- API Endpoints ---
    @app.get("/api/config")
    def get_config():
        return global_config

    def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
        """The full hash ? validate ? segment ? report pipeline for one video.

        This is the former synchronous /api/process body, unchanged in behavior, now run on
        the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
        the sync endpoint used to return (UI compatibility); the per-video observability
        report is still emitted in `finally:` and grafted as response["reports"].

        `progress` is an optional callable(stage: str) used to surface coarse job progress.
        Raises on unexpected internal errors ? the caller records those as a job failure
        (after this function has already restored the pre-run segments, see below).
        """
        notify = progress or (lambda stage: None)

        notify("hashing")
        video_id = compute_video_id(req.video_path)
        was_reingest = db_instance.get_video(video_id) is not None

        # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
        notify("validating")
        logger.info(f"Running validations for {req.video_path}...")
```

```
146         val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
147         failures = [r for r in val_results if not r.passed]
148
149         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
150         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
```

43. assistant (2026-06-20T16:57:15.650Z)

Let me continue reading the main processing function to see where results are stored:

44. user (2026-06-20T16:57:16.304Z)

```
150         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
151         seg_name = req.segmenter or default_seg
152         segmenter_actual = None
153         segmentation_ran = False
154         response: Optional[Dict[str, Any]] = None
155         try:
156             if failures:
157                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
158                 # status; it no longer destroys the video's prior good segments.
159                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
160                 response = {
161                     "video_id": video_id,
162                     "status": "failed",
163                     "message": "Video failed validation checks.",
164                     "results": [r.model_dump() for r in val_results],
165                 }
166                 return response
167
168             # 2. Run segmenter & indexer
169             logger.info("[API] Video passed checks. Segmenting and indexing...")
170             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
171
172             segmenter_cls = segmenter_registry.get(seg_name)
173             if not segmenter_cls:
174                 # Submit-time validation already 400s unknown names; this is the defensive
175                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
176                 available = list(segmenter_registry.list_available().keys())
177                 response = {
178                     "video_id": video_id,
179                     "status": "failed",
180                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
181                     "results": [r.model_dump() for r in val_results],
182                     "play_segments": [],
183                 }
184                 return response
185
186             logger.info(f"[API] Using segmenter: {seg_name}")
187             notify(f"segmenting ({seg_name})")
188             segmenter_actual = seg_name
189             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
190             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
191             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
192             segmenter = segmenter_cls(db_instance, global_config)
193             try:
```

```
                result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
            except Exception:
                # A raising segmenter must not leave half-written rows: restore the pre-run
                # state (same destroy-after-confirm contract as the Failure branch), then
let
                # the job worker record the failure.
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                raise
            segmentation_ran = True

            if isinstance(result, Failure):
                logger.error(f"[API] Segmenter failed: {result.message}")
                db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
                response = {
                    "video_id": video_id,
                    "status": "failed",
                    "message": f"Segmenter '{seg_name}' failed: {result.message}",
                    "results": [r.model_dump() for r in val_results],
                    "play_segments": [],
                }
                return response

            segments = result.value if hasattr(result, "value") else result
            notify("finalizing")
            _finalize_reingest(video_id, segments, play_wm, cand_wm)

            response = {
                "video_id": video_id,
                "status": "success",
                "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
```

```
        if not os.path.exists(req.video_path):
            raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}")
    def get_job(job_id: str):
        """Job state: {status, progress, result, reports}. `status` = execution state
        (queued|running|done|failed); the pipeline verdict is result["status"]."""
        job = db_instance.get_job(job_id)
        if not job:
            raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
        result = job.get("result")
        return {
            "job_id": job["id"],
            "status": job["status"],
            "progress": job.get("progress"),
            "video_id": job.get("video_id"),
            "error": job.get("error"),
            "result": result,
            "reports": (result or {}).get("reports"),
            "created_at": job.get("created_at"),
            "started_at": job.get("started_at"),
            "finished_at": job.get("finished_at"),
        }

    # --- Chunked upload (B2, PR-2) ----------------------------------------------------
    # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
in
    # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
via
    # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
    # sequential-part logic, and abort cleanup ? is unchanged.
    from backend.api import uploads as uploads_api

    app.include_router(uploads_api.router)


    # --- Highlight download (B3, PR-2) ------------------------------------------------

    @app.get("/api/highlights/{video_id}/{filename}")
    def download_highlight(video_id: str, filename: str):
        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
        (which only writes into output_dir; the browser previously got back an unreachable
        server-local path)."""
        try:
            name = safe_output_filename(filename)
```

```
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not db_instance.get_video(video_id):
            raise HTTPException(status_code=404, detail="Indexed video record not found.")
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        path = os.path.join(output_dir, name)
        try:
            path = resolve_under_root(path, output_dir)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        if not os.path.isfile(path):
            raise HTTPException(status_code=404,
                                detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
        return FileResponse(path, media_type=media, filename=name)


    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Reject path traversal / absolute output names (os.path.join would otherwise let an
        # absolute or '..'-laden output_filename write anywhere on disk).
        try:
            out_name = safe_output_filename(req.output_filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e

        # The configured shot-count floor (stitching.min_shot_count) applies on the API
        # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
        # metadata are exempt: the floor filters known counts only, so explicitly
        # requested manual/legacy segments can't silently vanish under the default floor.
        stitching = getattr(global_config, "stitching", None) or StitchingConfig()
        kept = []
        for s in matched_segments:
            meta = s.get("metadata") or {}
            sc = meta.get("shot_count") if isinstance(meta, dict) else None
            try:
                if sc is not None and int(sc) < stitching.min_shot_count:
                    continue
            except (TypeError, ValueError):
```

```
375                 pass  # unparseable count -> treat as unknown, keep
376             kept.append(s)
377         if len(kept) < len(matched_segments):
378             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
379                         f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
380         if not kept:
381             raise HTTPException(status_code=400,
382                                 detail=f"All {len(matched_segments)} matching segments fall
below "
383
f"stitching.min_shot_count={stitching.min_shot_count}.")
384
385         # Extract start/end time pairs
386         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
387
388         # Run compiler with the configured stitching paddings + optional branding watermark
389         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
390         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
391                                  end_padding=stitching.end_padding,
watermark=stitching.watermark)
392         try:
393             out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
394             return {"status": "success", "filepath": out_path}
395         except Exception as e:
396             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
397
398     # --- CLI Debug Utility & Orchestration ---
399     def run_cli_diagnose_clip(video_path: str, timestamp: float):
```

45. user (2026-06-20T16:57:16.992Z)

```
Found 2 files
backend\reporting\spec.yaml
backend\reporting\__init__.py
```

46. assistant (2026-06-20T16:57:19.235Z)

Now let me check the reporting module for report outputs:

47. user (2026-06-20T16:57:19.765Z)

```
1       """Per-video observability reports for the indexer.
2
3       `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4       no-op and the pipeline runs exactly as before. The single entrypoint
5       `emit_indexer_report(...)` is designed to be called from a `finally` block and
6       NEVER raises ? emitting a report must not be able to break a processing run.
7       """
8       from __future__ import annotations
9
10      from typing import Any, Dict, List, Optional
11
12      from . import run_signals  # no obsreport dependency ? always importable
13
14      try:  # soft dependency ? never hard-fail the pipeline on a reporting import
15          import obsreport  # noqa: F401
16          from . import harvest
17
18          OBSREPORT_AVAILABLE = True
19      except Exception:  # pragma: no cover - exercised only when obsreport is absent
```

```
20          OBSREPORT_AVAILABLE = False
21
22
23      def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
24          return bool((config or {}).get("report", {}).get("enabled", True))
25
26
27      def emit_indexer_report(
28          *,
29          db,
30          video_id: str,
31          video_path: Optional[str],
32          config: Optional[Dict[str, Any]],
33          val_results: List[Any],
34          val_context: Optional[Dict[str, Any]] = None,
35          segmenter_requested: Optional[str] = None,
36          segmenter_actual: Optional[str] = None,
37          was_reingest: bool = False,
38          segmentation_ran: bool = True,
39      ) -> Optional[Dict[str, str]]:
40          """Harvest + write the report. Returns the written file paths, or None if
41          reporting is unavailable/disabled/failed. Never raises."""
42          config = config or {}
43          # Accept either a plain dict or a typed config object (e.g.
backend.config.AppConfig).
44          # Normalize to a plain nested dict so harvest's dict access works uniformly.
45          if hasattr(config, "model_dump"):
46              try:
47                  config = config.model_dump(mode="json")
48              except Exception:
49                  config = {}
50          # Consume run signals unconditionally (even when reporting is off) so the
51          # process-local store never accumulates entries for finished runs.
52          signals = run_signals.pop(video_id)
53          if not OBSREPORT_AVAILABLE or not report_enabled(config):
54              return None
55          try:
56              play_segments = db.query_play_segments(video_id, {}) if db is not None else []
57              candidates = db.query_candidate_segments(video_id) if db is not None else []
58              rec = harvest.record_run(
59                  video_id=video_id,
60                  video_path=video_path,
61                  config=config,
62                  val_results=val_results,
63                  val_context=val_context,
64                  play_segments=play_segments,
65                  candidates=candidates,
66                  segmenter_requested=segmenter_requested,
67                  segmenter_actual=segmenter_actual,
68                  was_reingest=was_reingest,
69                  segmentation_ran=segmentation_ran,
70                  run_signals=signals,
71              )
72              output_dir = (config.get("report") or {}).get("output_dir")
73              res = rec.write(output_dir=output_dir)
74              return res.paths or None
75          except Exception:  # pragma: no cover - defensive; reporting must never break a run
76              return None
77
```

48. user (2026-06-20T16:57:20.087Z)

```
(Bash completed with no output)
```

49. assistant (2026-06-20T16:57:22.419Z)

Perfect! Now let me check the harvest module to see how reports are structured and written:

50. user (2026-06-20T16:57:22.845Z)

```
1       """Build an observability report for one indexer run.
2
3       Pure-ish harvest: given the validation results/context, the persisted segments,
4       and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5       *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6       so this module only records facts faithfully ? it does not decide warnings.
7
8       Imported only after `backend.reporting` confirms obsreport is installed, so it may
9       import obsreport freely.
10      """
11      from __future__ import annotations
12
13      import os
14      from pathlib import Path
15      from typing import Any, Dict, List, Optional
16
17      try:
18          import obsreport  # soft dep; only needed for RunRecorder / load_spec at runtime
19      except ImportError:
20          obsreport = None  # type: ignore[assignment]
21
22      _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
23      _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
24
25
26      def load_spec() -> "obsreport.ReportSpec":
27          return obsreport.load_spec(_SPEC_PATH)
28
29
30      # --- media metadata ------------------------------------------------------
31      def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
32          """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
33          distinguishes a real probe from the absence of one (the fallback footgun)."""
34          meta: Dict[str, Any] = {"fps_source": "unknown"}
35          if video_path and os.path.exists(video_path):
36              try:
37                  meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
38              except OSError:
39                  pass
40
41          ffmeta = (val_context or {}).get("ffprobe_meta") or {}
42          streams = ffmeta.get("streams") or []
43          fmt = ffmeta.get("format") or {}
44          if streams:
45              s = streams[0]
46              w, h = s.get("width"), s.get("height")
47              meta["width"], meta["height"] = w, h
48              if w and h:
49                  meta["resolution"] = f"{w}x{h}"
50              fps_str = s.get("r_frame_rate", "0/1")
51              try:
52                  num, den = (int(x) for x in fps_str.split("/"))
53                  if den:
54                      meta["fps"] = round(num / den, 2)
55                      meta["fps_source"] = "probed"
56              except (ValueError, ZeroDivisionError):
57                  pass
58              if s.get("codec_name"):
59                  meta["container"] = fmt.get("format_name") or s.get("codec_name")
60          if fmt.get("duration"):
```

```
61              try:
62                  meta["duration_s"] = round(float(fmt["duration"]), 1)
63              except (ValueError, TypeError):
64                  pass
65          if fmt.get("format_name"):
66              meta["container"] = fmt.get("format_name")
67
68          # Audio readiness signal (for Input Quality UX extension per review item 2).
69          # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
70          # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
71          # We only record presence here (cheap, from ffprobe already in context); the actual
72          # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
calibration
73          # tool for now.
74          audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]
75          meta["audio_present"] = len(audio_streams) > 0
76          return meta
77
78
79      # --- stage builders ----------------------------------------------------------
80      def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
81          with rec.stage("validation") as st:
82              total = len(val_results)
83              passed = sum(1 for r in val_results if getattr(r, "passed", False))
84              st.add_metric("checks_passed", passed)
85              st.add_metric("checks_total", total)
86              failures = [r for r in val_results if not getattr(r, "passed", True)]
87              for r in failures:
88                  st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r,
'message', '')}")
89              st.status = "error" if failures else ("ok" if total else "not_run")
90
91
92      def _segmentation_stage(
93          rec: "obsreport.RunRecorder",
94          play_segments: List[Dict[str, Any]],
95          candidates: List[Dict[str, Any]],
96          segmenter_requested: Optional[str],
97          segmenter_actual: Optional[str],
98          config_default: Optional[str],
99          was_reingest: bool,
100         ran: bool,
101     ) -> None:
102         with rec.stage("segmentation") as st:
103             if not ran:
104                 st.status = "not_run"
105             seg_count = len(play_segments)
106             total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
107             detector_tag = None
108             if play_segments:
109                 detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
110             st.add_metric("candidate_windows", len(candidates))
111             st.add_metric("segment_count", seg_count, audience="both")
112             st.add_metric("total_play_s", total_play, unit="s")
113             # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
114             # pick it, and it differs from config's default (the genuine surprise case).
115             # An explicit --segmenter override or a halted run is not a divergence.
116             matches_default = True
117             if ran and segmenter_actual and config_default:
118                 matches_default = (segmenter_actual == config_default)
119             st.details.update({
120                 "segmenter_requested": segmenter_requested,
121                 "segmenter_actual": segmenter_actual,
122                 "config_default": config_default,
123                 "segmenter_explicitly_requested": segmenter_requested is not None,
```

```
124                 "matches_config_default": matches_default,
125                 "was_reingest": was_reingest,
126                 "detector": detector_tag,
127             })
128             # Surface metadata trustworthiness at the data level (not just in a warning):
129             # the motion baseline fabricates per-segment metadata; everything else measures
it.
130             if ran and segmenter_actual:
131                 st.details["metadata_source"] = (
132                     "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
133                     else "measured (detector/AI)"
134                 )
135
136
137     def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
138                   play_segments: List[Dict[str, Any]], ran: bool,
139                   run_signals: Optional[Dict[str, Any]] = None) -> None:
140         """Only meaningful for AI-based segmenters. Records whether the provider key was
141         present ? the signal the silent_provider_noop rule keys on ? plus any in-process
142         run signals (backend.reporting.run_signals), e.g. oversize chunk refusals."""
143         ai_based = segmenter_actual in _AI_SEGMENTERS
144         provider = (config.get("ai_provider") or "gemini")
145         key_env = f"{provider.upper()}_API_KEY"
146         with rec.stage("ai_handoff") as st:
147             if not ran or not ai_based:
148                 st.status = "not_run" if not ran else "skipped"
149             st.details.update({
150                 "ai_based": ai_based,
```

51. user (2026-06-20T16:57:23.401Z)

```
C:\Users\avidu\Projects\badminton-highlight-indexer\output/ab_experiment_n6.json
C:\Users\avidu\Projects\badminton-highlight-indexer\output/adarsh_avi_full_vs_human.csv
C:\Users\avidu\Projects\badminton-highlight-
indexer\output/adarsh_avi_singles_golden_features.json
C:\Users\avidu\Projects\badminton-highlight-indexer\output/adarsh_avi_vs_human.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\output/adarsh_baseline_gated.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\output/adarsh_baseline_segments.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\output/adarsh_gemini_fullvideo.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\output/adarsh_gemini_refined.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\output/adarsh_gemini_smoke.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\output/adarsh_min5s_segments.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\output/adarsh_tuned_segments.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\output/Badminton_BXH_2.rallies.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\output/Badminton_BXH_2_golden_features.json
C:\Users\avidu\Projects\badminton-highlight-indexer\output/Boxhill_Doubles.rallies.csv
C:\Users\avidu\Projects\badminton-highlight-indexer\output/Boxhill_Doubles_golden_features.json
C:\Users\avidu\Projects\badminton-highlight-indexer\output/chunks/3d19ef835a69be93acdd75ef636f34
5a/chunk_3d19ef835a69be93acdd75ef636f345a_3.mp4
C:\Users\avidu\Projects\badminton-highlight-indexer\output/chunks/74affb36e15e172130cea5c451e4a7
28/chunk_74affb36e15e172130cea5c451e4a728_0.mp4
C:\Users\avidu\Projects\badminton-highlight-
indexer\output/chunks/chunk_018c2e2c3ce3086c3badfa1a9e5bc5c9_1.mp4
C:\Users\avidu\Projects\badminton-highlight-
indexer\output/chunks/chunk_e0168f050899da4de75bcd311b80131b_5.mp4
C:\Users\avidu\Projects\badminton-highlight-indexer\output/e2e_reel.mp4
```

52. assistant (2026-06-20T16:57:25.844Z)

Excellent! Now let me compile the complete mapping. Let me search for a few more key items:

53. user (2026-06-20T16:57:27.030Z)

```
No matches found
```

54. user (2026-06-20T16:57:27.841Z)

```
total 5049296
drwxr-xr-x 1 avidu 197609          0 Jun 20 15:49 .
drwxr-xr-x 1 avidu 197609          0 Jun 20 20:48 ..
-rw-r--r-- 1 avidu 197609     241664 Jun 19 17:27 19june.db
-rw-r--r-- 1 avidu 197609      31432 Jun 19 16:02 19june_extract.log
-rw-r--r-- 1 avidu 197609      22924 Jun 19 17:28 19june_extract_rerun.log
-rw-r--r-- 1 avidu 197609       3032 Jun 19 18:48 19june_local_vs_gemini.md
drwxr-xr-x 1 avidu 197609          0 Jun 20 00:17 19june_work
-rw-r--r-- 1 avidu 197609       1970 Jun 17 13:39 Badminton_BXH_2.rallies.csv
-rw-r--r-- 1 avidu 197609       3173 Jun 17 15:06 Badminton_BXH_2_golden_features.json
-rw-r--r-- 1 avidu 197609       1915 Jun 18 11:49 Boxhill_Doubles.rallies.csv
-rw-r--r-- 1 avidu 197609      17415 Jun 18 12:26 Boxhill_Doubles_golden_features.json
-rw-r--r-- 1 avidu 197609 1207870728 Jun 13 15:03 GX010125_highlights.mp4
-rw-r--r-- 1 avidu 197609 2083468872 Jun 13 20:19 GX010125_highlights_min5.mp4
drwxr-xr-x 1 avidu 197609          0 Jun 14 10:48 GX010125_run
-rw-r--r-- 1 avidu 197609       2286 Jun 16 22:27 GX010128.rallies.csv
-rw-r--r-- 1 avidu 197609       6386 Jun 16 22:32 GX010128_golden_features.json
-rw-r--r-- 1 avidu 197609       1489 Jun 18 10:13 GX010137.rallies.csv
-rw-r--r-- 1 avidu 197609      16267 Jun 18 10:22 GX010137_golden_features.json
-rw-r--r-- 1 avidu 197609       1274 Jun 18 06:49 GX010141.rallies.csv
```

55. assistant (2026-06-20T16:57:29.663Z)

Perfect! Now let me create a comprehensive summary. Let me gather a bit more detail on the schema fields:

56. user (2026-06-20T16:57:30.216Z)

```
110             """Create the videos / play_segments / events base tables (idempotent)."""
111             # Videos Table
112             cursor.execute("""
113                 CREATE TABLE IF NOT EXISTS videos (
114                     id TEXT PRIMARY KEY,
115                     filepath TEXT NOT NULL,
116                     processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
117                     status TEXT NOT NULL,
118                     validation_results TEXT
119                 )
120             """)
121
122             # Play Segments Table (Extensible metadata JSON)
123             cursor.execute("""
124                 CREATE TABLE IF NOT EXISTS play_segments (
125                     id INTEGER PRIMARY KEY AUTOINCREMENT,
126                     video_id TEXT NOT NULL,
127                     start_time REAL NOT NULL,
128                     end_time REAL NOT NULL,
129                     duration REAL NOT NULL,
130                     server TEXT,
131                     receiver TEXT,
132                     metadata TEXT,
133                     FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
134                 )
135             """)
136
137             # Events Table
138             cursor.execute("""
139                 CREATE TABLE IF NOT EXISTS events (
140                     id INTEGER PRIMARY KEY AUTOINCREMENT,
141                     segment_id INTEGER NOT NULL,
```

```
                    timestamp REAL NOT NULL,
                    type TEXT NOT NULL,
                    player TEXT,
                    details TEXT,
                    FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
                )
            """)

        def _migrate_to_v1(self, cursor):
            # v1: raw candidate detections (pre-confirmation), detector-agnostic.
            # Common fields are columns; detector-specific metrics go in `stats` JSON,
            # so a new segmenter reuses this table by setting its own `detector`/`stats`.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS candidate_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    detector TEXT NOT NULL,
                    chunk_index INTEGER,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    confidence REAL,
                    stats TEXT,
                    detection_params TEXT,
                    confirmed_segment_id INTEGER,
                    human_label TEXT,
                    notes TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                    FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
                )
            """)
            cursor.execute("""
                CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                    ON candidate_segments(video_id, detector)
            """)
            cursor.execute("PRAGMA user_version = 1")

        def _migrate_to_v2(self, cursor):
            # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
            # /api/process request. `status` is the EXECUTION state of the job
            # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
            # that failed validation) lives inside `result` JSON as result["status"].
            # `result` is the full sync-era response dict (incl. report paths), so the
            # observability report is the job's artifact, per the plan.
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS jobs (
                    id TEXT PRIMARY KEY,
                    video_path TEXT NOT NULL,
                    video_id TEXT,
                    segmenter TEXT,
                    player_pool TEXT,
                    max_frames INTEGER,
                    status TEXT NOT NULL DEFAULT 'queued',
                    progress TEXT,
                    error TEXT,
                    result TEXT,
                    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    started_at TIMESTAMP,
                    finished_at TIMESTAMP
                )
            """)
            cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
            cursor.execute("PRAGMA user_version = 2")
```

```
206
207         def _migrate_to_v3(self, cursor):
208             # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
209             # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
due
210             # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
211             # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
212             # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
213             # ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
214             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
215             self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN next_poll_at
TIMESTAMP")
216             cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
217             cursor.execute("PRAGMA user_version = 3")
218
219         def _migrate_to_v4(self, cursor):
220             # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a NULLABLE
221             # `user_id` to videos + candidate_segments so a future per-user / multi-tenant
222             # path can scope rows to an owner. **NULL = local install = byte-identical to
223             # today** ? every existing row stays NULL and every existing query (which never
224             # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows survive;
225             # the ADD COLUMNs are applied idempotently so an interrupted upgrade can re-run
them.
226             # Additive ONLY: no served-behavior change rides on this slice.
227             self._add_column_idempotent(cursor, "ALTER TABLE videos ADD COLUMN user_id
TEXT")
228             self._add_column_idempotent(cursor, "ALTER TABLE candidate_segments ADD COLUMN
user_id TEXT")
229             # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast path free
230             # of index churn while making the eventual per-user lookups cheap.
231             cursor.execute(
232                 "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
233                 "WHERE user_id IS NOT NULL")
234             cursor.execute(
235                 "CREATE INDEX IF NOT EXISTS idx_candidates_user "
236                 "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
237             cursor.execute("PRAGMA user_version = 4")
238
239         def _migrate_to_v5(self, cursor):
240             # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per (user_id,
241             # version): the resolved per-user `fusion_config` overlay + an INLINE
242             # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
243             # and provenance that earned it. `is_active` pins the one row the serving bridge
244             # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`). This
slice
245             # only persists/loads it ? NOTHING reads it on the served path yet.
246             #
247             #   user_id              ? owner of the model (NULL allowed = the local install)
248             #   version              ? monotonic per-user model version
249             #   baseline_version     ? the shared baseline this was fit against (upgrade
switch)
250             #   feature_schema_hash  ? fingerprint of the feature set; MINOR-vs-MAJOR re-fit
gate
251             #   fusion_config        ? JSON per-user FusionConfig overlay (cue_flags /
thresholds)
252             #   classifier_json      ? JSON LogisticRegression.to_dict() (None = config-only
model)
253             #   promotion_evidence   ? JSON per_user_gate PromotionDecision (why it was
promoted)
254             #   n_videos_at_fit      ? held-out-user corpus size when fit (min_n trust floor
input)
255             #   is_active            ? the one resolved row for this user (partial-unique
```

```
below)
256             cursor.execute("""
257                 CREATE TABLE IF NOT EXISTS user_models (
258                     id INTEGER PRIMARY KEY AUTOINCREMENT,
259                     user_id TEXT,
260                     version INTEGER NOT NULL DEFAULT 1,
261                     baseline_version TEXT,
262                     feature_schema_hash TEXT,
263                     fusion_config TEXT,
264                     classifier_json TEXT,
265                     promotion_evidence TEXT,
266                     n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
267                     is_active INTEGER NOT NULL DEFAULT 0,
268                     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
269                 )
270             """)
271             # One model version per user (NULLs compare distinct in SQLite, so the local
272             # install can still carry multiple versioned rows; the active-row guard below
273             # is what enforces a single served model).
274             cursor.execute("""
275                 CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
276                     ON user_models(user_id, version)
277             """)
278             # At most ONE active model per user: a partial-unique index over the active
rows.
279             # (SQLite treats NULL user_id rows as distinct, so the local install's single
280             # active row is still guarded ? there is one local user.)
281             cursor.execute("""
282                 CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
283                     ON user_models(user_id) WHERE is_active = 1
284             """)
285             cursor.execute("PRAGMA user_version = 5")
286
287         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
288             """Register or update a video row WITHOUT touching its child segments.
289
```

57. assistant (2026-06-20T16:57:32.341Z)

Now I have all the information I need to compile the comprehensive mapping. Let me create the final report:

58. user (2026-06-20T16:58:06.234Z)

Structured output provided successfully

59. assistant (2026-06-20T16:58:14.005Z)

Perfect! I've completed a comprehensive mapping of all per-video artifact write sites and database metadata for the badminton-highlight-indexer project. Here's a summary of what was found:

Key Findings

```
**Database Core (SQLite: sports_indexer.db)**
- 8 tables total, schema version 5
- All rally metadata, detection stats, and job state stored as pure metadata
- **videos**: source path + status (no paths to outputs)
- **play_segments**: detected rallies (start/end times, duration, metadata JSON)
- **candidate_segments**: pre-confirmation detections per detector
- **events**: per-rally events (e.g., smash, foul)
- **jobs**: async job state + result JSON (contains report paths)
- **user_models**: per-user ML models (inline classifiers, no file artifacts)
```

Artifact Write Sites

1. **EVAL outputs**: CSVs from trajectory ? windowing pipeline
   - `output/wasb_tuned_segments.csv` (hardcoded default)
   - `output/wasb_tuned_segments_vs_golden.csv` (comparison)

2. **Highlight reels**: Video compilation
   - `output/<basename>.mp4` (VideoCompiler writes to output_dir)
   - Clips created in tempdir during stitching, cleaned after concat

3. **Run logs**: Streamed via Python logging (not auto-persisted)
   - Manual capture: `> output/<name>_extract.log`

4. **Observability reports**: Per-video JSON (via obsreport library)
   - `output/<video_id>_report.json` (soft dependency)

5. **Job results**: Stored in `jobs.result` (JSON column in DB)
   - Contains: validation results, play_segments, report paths

Key Schema Fields (Metadata vs. File Paths)
- **Metadata fields**: timestamps, confidence scores, detector names, shot counts, validation results (all stored as JSON)
- **File path references**: Only in videos.filepath (source) and jobs.result['reports'] (output artifacts)
- **Keying**: All per-video records keyed by video_id (MD5 hash of source filepath)

All details are documented in the StructuredOutput above with file:line references for each artifact location.